

Corso di Laurea in Ingegneria e Scienze Informatiche

**Generazione automatica di dataset
text-to-SPARQL per fine-tuning di
LLM su knowledge graph
clinico-oncologico**

Tesi di laurea in:
PROGRAMMAZIONE

Relatore

Prof.ssa Antonella Carbonaro

Candidato

Matteo Tonelli

Sommario

Il lavoro proposto si colloca nel progetto Cancer Virtual Lab (CVL), il quale raccoglie i dati clinici oncologici dell'IRCCS IRST "Dino Amadori" in un Knowledge Graph interrogabile tramite SPARQL¹. La natura formale di SPARQL rappresenta però una barriera per il personale sanitario e un'interfaccia in linguaggio naturale (Text-to-SPARQL) permetterebbe di abbattere tale barriera, ma richiederebbe un modello linguistico addestrato sull'ontologia specifica di CVL, per la quale non esistono dataset di addestramento pubblici.

Questo lavoro progetta e implementa una pipeline automatizzata a due stadi per la generazione di un dataset Text-to-SPARQL sintetico, garantendo correttezza e varietà. Il primo stadio esplora lo schema del grafo e campiona combinazioni di vincoli sintatticamente corrette, coprendo diverse entità organizzate in gruppi di priorità basati sulla centralità clinica. Il secondo stadio applica un meccanismo di mascheramento dei valori letterali e un ciclo iterativo di parafrasi guidata per generare varianti in più registri stilistici distinti, aumentando la ricchezza linguistica senza alterare la semantica delle query.

La pipeline ha prodotto in fase di testing un dataset di 14.021 esempi, strutturato con una rigorosa separazione per record sorgente tra gli insiemi di addestramento, validazione e test per evitare data leakage. Un modello **Gemma3:270M**, sottoposto a fine-tuning mediante LoRA su questo dataset, ha raggiunto un exact match del 46% sull'insieme di test in-distribution, individuando il checkpoint ottimale tramite validation loss e superando nettamente il full fine-tuning (18%). L'analisi degli errori residui ha mostrato un tasso dello 0% di query sintatticamente non valide, dimostrando che il modello apprende saldamente la grammatica SPARQL. Vengono infine analizzati i problemi di qualità emersi nel dataset, delineando le relative strategie di risoluzione.

¹ [<https://www.w3.org/TR/sparql11-query/>] (<https://www.w3.org/TR/sparql11-query/>)

Indice

Sommario	iii
1 Introduzione	1
2 Background	3
2.1 Cancer Virtual Lab	4
2.1.1 La Sfida della Frammentazione dei Dati Clinici	5
2.1.2 Standard di Interoperabilità: da HL7 FHIR a RDF	5
2.1.3 L'Assistente Virtuale e la Necessità di Dataset Dedicati . . .	6
2.2 Knowledge Graph, RDF e Strumenti SPARQL	7
2.2.1 Dai Dati Strutturati ai Knowledge Graph	8
2.2.2 Resource Description Framework (RDF)	8
2.2.3 SPARQL	9
2.2.4 Validazione Sintattica Determinista con RDFLib	10
2.3 Architettura Software e Pattern	10
2.3.1 Il Pattern Architetturale Mixin	10
2.3.2 Prototipazione e Interfacce Reattive con Streamlit	11
2.4 Lavori Correlati e Benchmark di Text-to-SPARQL	12
2.4.1 Approcci Basati su Template Rigidi e Corpora Sintetici . . .	12
2.4.2 Sintesi Guidata da Regole e Grammatiche Formali	13
2.4.3 Generazione Indotta da Large Language Model	13
2.4.4 Motivazioni della Scelta Metodologica nel Contesto CVL . .	14
2.5 Fine-tuning di Large Language Model	14
2.5.1 Adattamento supervisionato (SFT)	14
2.5.2 Full Fine-Tuning	15
2.5.3 Tecniche di adattamento efficiente (PEFT, LoRA e QLoRA)	15
2.5.4 Funzione di perdita e formato delle istruzioni	16
3 Analisi	17
3.1 Obiettivi	17
3.2 Vincoli	18

3.3	Requisiti funzionali	19
3.4	Requisiti non funzionali	19
4	Progettazione	21
4.1	Analisi dei problemi	21
4.2	Design del generatore	22
4.2.1	La pipeline di generazione	23
4.2.2	Modello del dominio	24
4.2.3	Sistema dei parametri	26
4.2.4	Meccanismo di estensione incrementale	27
4.3	Design dell'augmentation	28
4.3.1	Mascheramento dei valori letterali e strategie di preservazione	28
4.3.2	Generazione iterativa e validazione	29
4.3.3	Varietà stilistica	31
5	Implementazione	33
5.1	Implementazione del generatore	33
5.1.1	Architettura a mixin	33
5.1.2	Costruzione e indicizzazione della mappa del grafo	36
5.1.3	Costruzione delle route	38
5.1.4	Pianificazione: stima combinatoria e campionamento	39
5.1.5	Esecuzione sequenziale e parallela	41
5.1.6	Costruzione dello scheletro: percorso online e offline	43
5.1.7	Riempimento dei valori	46
5.1.8	Generazione del prompt e tracciamento degli span	47
5.1.9	Controlli di qualità e deduplicazione	49
5.1.10	Generazione manuale a entità singola	50
5.1.11	Configurazione dei tier da file esterno	52
5.1.12	Split del dataset	53
5.2	Implementazione dell'augmentation	54
5.2.1	Mascheramento	55
5.2.2	Mascheramento deterministico dei valori	55
5.2.3	Semplificazione booleana e generazione degli hint	56
5.2.4	Suddivisione in chunk per record complessi	58
5.2.5	Ingegneria dei prompt e direttive di riformulazione	59
5.2.6	Generazione iterativa: stile, retry e fallback	61
5.2.7	Validazione: due funzioni per due modalità	63
5.2.8	Rotazione degli stili e memoria anti-ripetizione	64
5.2.9	Statistiche di qualità e resilienza	65
5.2.10	La modalità senza mascheramento	67

6	Valutazione	69
6.1	Testing	69
6.1.1	Problemi di qualità riscontrati e diagnosi delle cause	69
6.1.2	Confronto tra generazione con e senza mascheramento	71
6.1.3	Statistiche del dataset ed analisi delle distribuzioni	73
6.1.4	Metodologia	75
6.1.5	Selezione del checkpoint e prevenzione dell'overfitting	75
6.1.6	Confronto tra le configurazioni di addestramento	76
6.1.7	Tassonomia e analisi diagnostica degli errori	78
7	Conclusioni e sviluppi futuri	81
7.1	Sviluppi futuri	81
7.1.1	Risoluzione dei problemi di qualità	81
7.1.2	Altre direzioni di sviluppo	82
7.1.3	Conclusioni	83
	Bibliografia	85

Elenco delle figure

4.1	Architettura macro ad alto livello della pipeline dati: flusso sequenziale dal Knowledge Graph fino alla validazione del dataset finale.	23
4.2	Modello di dominio semplificato, si noti Patient come nodo centrale	25
4.3	Modello esemplificativo dei parametri filtrabili	26
6.1	Distribuzione delle occorrenze per i quattro problemi quantificabili (il problema di sintassi non naturale, individuato tramite ispezione manuale, non compare in figura).	70
6.2	Distribuzione delle varianti accettate tra i dieci registri stilistici nell'insieme di addestramento.	74
6.3	Andamento della train e validation loss durante le 15 000 iterazioni.	76
6.4	Confronto dell'Exact Match sulle configurazioni testate.	77
6.5	Distribuzione percentuale delle 54 predizioni non corrispondenti per tipologia di errore sul checkpoint ottimale (iterazione 7 000).	78

Elenco dei listati

5.1	Composizione della classe principale del generatore tramite mixin. .	33
5.2	Controllo dell'interruzione durante la generazione.	34
5.3	Costanti di classe utilizzate dal generatore.	35
5.4	Struttura della mappa del grafo memorizzata in <code>graph_map</code>	36
5.5	Visita in profondità della mappa del grafo per la costruzione delle route disponibili a partire da un'entità radice.	38
5.6	Campionamento casuale di un insieme di route indipendenti tra loro per un singolo record.	41
5.7	Scelta tra esecuzione sequenziale e parallela in base al numero di worker configurati.	42
5.8	Configurazione condivisa usata nei run di produzione, per entrambe le liste di tier.	43
5.9	Lo scheletro appena ricostruito per i terminal <code>gender</code> e <code>birth_date</code> , ancora privo di valori.	46
5.10	Riempimento di un nodo dello scheletro con valori concreti, distinguendo tra intervalli e insiemi discreti.	46
5.11	Lo stesso scheletro di listato 5.9, dopo il riempimento: elenco discreto per il campo categoriale, intervallo per quello di tipo data. .	47
5.12	Costruzione del prompt segmento per segmento, con tracciamento diretto della posizione di ciascun valore letterale.	48
5.13	Verifica della presenza dei vincoli nella query SPARQL.	49
5.14	Validazione sintattica condizionata della query SPARQL.	50
5.15	Esempio minimo di configurazione di un tier da file JSON esterno. .	52
5.16	Suddivisione del dataset in train, validation e test, stratificata per entità di origine.	53
5.17	Generazione dei placeholder escludendo i caratteri ad alta frequenza di allucinazione e mascheramento basato su posizioni.	56
5.18	Trascrizione proposizionale ed eliminazione preventiva dei placeholder booleani.	56
5.19	Suddivisione in chunk del testo mascherato	58

5.20	Struttura dei template di prompt e funzioni per l'iniezione dinamica del contesto di feedback e storico.	60
5.21	Ciclo di generazione e validazione iterativa per un singolo chunk con gestione del fallback.	61
5.22	La funzione di validazione condivisa da chunk e varianti complete, con soglia di somiglianza adattiva.	63
5.23	Assegnazione ciclica degli stili e tracciamento a finestra delle formulazioni e delle aperture recenti.	64
5.24	Calcolo della resa complessiva del processo di generazione e serializzazione delle metriche di qualità.	66
5.25	Implementazione della modalità baseline senza mascheramento per il confronto sperimentale.	67

Capitolo 1

Introduzione

Negli ultimi anni, l'avvento dell'Intelligenza Artificiale Generativa (GenAI) e dei *Large Language Models* sta trasformando radicalmente il modo in cui le persone interagiscono con la tecnologia, aprendo scenari inediti per la creazione di interfacce avanzate in linguaggio naturale. Nel dominio della sanità e della ricerca biomedica, dove la complessità delle informazioni e la frammentazione delle basi di conoscenza rappresentano da sempre un limite operativo, l'adozione di soluzioni basate su GenAI offre l'opportunità di abbattere le barriere tecniche di accesso al dato. Consentire a medici, clinici e ricercatori di interrogare i sistemi informativi sanitari ponendo semplici domande nel proprio linguaggio naturale significa velocizzare i processi diagnostici, facilitare l'estrazione di evidenze scientifiche e restituire al personale sanitario un ruolo centrale e diretto nella fruizione delle informazioni, senza la mediazione di intermediari informatici o la necessità di apprendere linguaggi di interrogazione complessi.

In questo contesto si inserisce il presente lavoro di tesi, che affronta e risolve l'assenza di un dataset di addestramento specifico per l'interrogazione in linguaggio naturale del *knowledge graph* del Cancer Virtual Lab. A tale scopo, è stata progettata, sviluppata e validata una pipeline automatica a due stadi per la generazione di dati sintetici, la cui efficacia è stata poi dimostrata empiricamente. Il primo stadio della pipeline esplora il grafo di conoscenza per costruire query SPARQL deterministiche e sintatticamente valide, mentre il secondo stadio, fondato su tecniche di *data augmentation*, declina tali strutture in un ampio spettro di

forme conversazionali fluide e stilisticamente varie. Per certificare in modo inconfutabile l'efficacia metodologica della pipeline e far fronte alle limitazioni hardware dell'ambiente di sviluppo della tesi, il dataset generato è stato impiegato per effettuare il *fine-tuning* in locale di un modello linguistico ultra-compatto. Riuscire a infondere la complessa grammatica formale e la semantica dell'ontologia medica in un'architettura così minimale ha costituito lo "stress test" definitivo per la qualità del corpus sintetico, dimostrando come una struttura di dati estremamente curata sia in grado di colmare le limitazioni capacitive della rete neurale e ponendo basi eccellenti per il futuro impiego su modelli di grande scala all'interno dell'infrastruttura clinica.

Il presente elaborato è organizzato in modo da guidare il lettore attraverso tutte le fasi logiche e tecniche della ricerca. Il capitolo 2 fornisce il contesto teorico, introducendo i concetti fondamentali sui grafi di conoscenza, sul linguaggio SPARQL e sulle architetture dei modelli linguistici. Il capitolo 3 definisce i requisiti funzionali, i vincoli di riservatezza e gli obiettivi architetturali del progetto, mentre il capitolo 4 illustra nel dettaglio il design della pipeline a due stadi. Successivamente, il capitolo 5 scende nei dettagli operativi dello sviluppo, analizzando le sfide affrontate nella manipolazione dell'ontologia e nella generazione stilistica. Infine, il capitolo 6 presenta la valutazione sperimentale dell'addestramento, analizzando le prestazioni e le metriche di errore, prima di giungere alle conclusioni che riassumono i traguardi raggiunti e delineano le direzioni di sviluppo future.

Capitolo 2

Background

Questo capitolo fornisce il quadro teorico, metodologico e tecnologico necessario a comprendere il contesto applicativo e le decisioni progettuali alla base dell'intero lavoro di tesi. La generazione automatica di dataset sintetici per la traduzione da linguaggio naturale a linguaggi formali di interrogazione (*Text-to-SPARQL*) si colloca infatti all'intersezione tra diverse discipline informatiche: l'ingegneria del Web Semantico, la modellazione di dati clinici e il *Natural Language Processing* (NLP) applicato ai modelli linguistici di grandi dimensioni (*Large Language Model*, LLM).

Per fornire una visione più strutturata, il capitolo è organizzato in cinque sezioni interconnesse:

- **Cancer Virtual Lab (sezione 2.1):** descrive la piattaforma del Cancer Virtual Lab (CVL), illustrando le sfide legate alla frammentazione dei dati oncologici, l'adozione dello standard internazionale HL7 FHIR e le motivazioni cliniche che guidano la realizzazione di un'interfaccia di interrogazione in linguaggio naturale.
- **Knowledge Graph, RDF e Strumenti SPARQL (sezione 2.2):** introduce le nozioni fondamentali sui Knowledge Graph e sul modello di rappresentazione a triple RDF (*Resource Description Framework*). Viene inoltre approfondito il linguaggio di interrogazione SPARQL, insieme al ruolo degli

strumenti software locali (in particolare la libreria Python `rdflib`)¹ utilizzati per la validazione sintattica deterministica delle query.

- **Architettura Software e Pattern (sezione 2.3):** analizza le scelte tecnologiche e i pattern di progettazione software adottati nell'ingegneria del sistema, tra cui l'architettura a *Mixin* per la composizione modulare delle responsabilità e il framework *Streamlit*² per la prototipazione di interfacce interattive di monitoraggio e controllo.
- **Lavori Correlati e Benchmark di Text-to-SPARQL (sezione 2.4):** presenta una rassegna dello stato dell'arte sui dataset di riferimento per il compito di traduzione Text-to-SPARQL, evidenziando le limitazioni dei benchmark generali (come LC-QuAD) quando applicati a domini specialistici e analizzando le criticità legate all'uso diretto di modelli generativi senza meccanismi di contenimento delle allucinazioni.
- **Fine-Tuning di Modelli Linguistici nel Dominio Clinico (sezione 2.5):** approfondisce le tecniche di adattamento supervisionato (SFT), ponendo a confronto il *Full Fine-Tuning* con le strategie di addestramento efficiente *Parameter-Efficient Fine-Tuning* (PEFT) come LoRA e QLoRA. La sezione motiva inoltre la scelta di modelli ultra-compatti per l'esecuzione locale su dati sensibili e descrive l'adozione del mascheramento della loss sui token di input (*Completion-Only Data Collator*).

2.1 Cancer Virtual Lab

Il lavoro descritto in questa tesi si inserisce direttamente nel contesto applicativo e tecnologico del progetto *Cancer Virtual Lab* (CVL) [AAG⁺25], una piattaforma avanzata di *AI Semantica* progettata per supportare la ricerca oncologica e la pratica clinica guidata dai dati. Il progetto è frutto di una collaborazione multidisciplinare tra enti di ricerca clinica e tecnologica, tra cui l'IRCCS Istituto Roma-

¹[\[https://rdflib.readthedocs.io/en/stable/\]](https://rdflib.readthedocs.io/en/stable/) (<https://rdflib.readthedocs.io/en/stable/>)

²[\[https://streamlit.io/\]](https://streamlit.io/) (<https://streamlit.io/>)

gnolo per lo Studio dei Tumori (IRST) "Dino Amadori" di Meldola e l'Università di Bologna.

2.1.1 La Sfida della Frammentazione dei Dati Clinici

Nel contesto della pratica oncologica reale, le informazioni cliniche e diagnostiche dei pazienti risultano intrinsecamente eterogenee, complesse e frammentate. I dati sanitari si trovano tipicamente segregati all'interno di silos informativi autonomi e non comunicanti:

- *Cartelle Cliniche Elettroniche (EHR)*: contenenti la storia anamnestica, le visite oncologiche e il tracciamento dei ricoveri;
- *Sistemi Informativi di Anatomia Patologica*: contenenti i referti istologici e la caratterizzazione molecolare del tumore;
- *Registri Farmaceutici e Somministrazioni*: contenenti il dettaglio dei protocolli chemioterapici, immunoterapici e delle linee di trattamento;
- *Laboratori di Analisi e Diagnostica per Immagini*: contenenti la serie storica degli esami ematochimici e le valutazioni strumentali di risposta lesionale.

Tale frammentazione costituisce una barriera sostanziale alla riutilizzabilità secondaria dei dati per scopi di ricerca clinica. Attività fondamentali come l'identificazione di coorti di pazienti con specifiche caratteristiche molecolari, la valutazione dell'efficacia di terapie neoadiuvanti o l'analisi dell'insorgenza di tossicità severa ed eventi avversi richiedono un processo lungo e oneroso di estrazione e riconciliazione manuale delle informazioni.

2.1.2 Standard di Interoperabilità: da HL7 FHIR a RDF

Per superare la rigidità dei silos informativi, la piattaforma CVL adotta un'architettura integrata di trasformazione semantica dei dati [CGR⁺25]. Il punto d'ingresso per la standardizzazione delle informazioni cliniche è rappresentato dallo standard internazionale Fast Healthcare Interoperability Resources (FHIR), sviluppato dall'organizzazione HL7. FHIR definisce un insieme di modelli di dati modulari

e tipizzati, denominati *Risorse* (es. `Patient`, `Condition`, `MedicationStatement`, `Observation`), progettati per facilitare lo scambio sicuro di informazioni tra sistemi sanitari eterogenei.

Tuttavia, mentre FHIR eccelle come formato di interscambio operativo via API RESTful, le interrogazioni analitiche su relazioni complesse e multi-livello traggono maggiore beneficio da modelli orientati ai grafi. Per questo motivo, la pipeline di CVL mappa e trasforma le risorse FHIR anonimizzate in un Knowledge Graph interoperabile basato sullo standard W3C Resource Description Framework (RDF) [CGR⁺25]. Questo processo ha consentito la costruzione di una base di conoscenza semantica integrata comprendente i dati clinici, diagnostici e terapeutici di oltre 35.000 pazienti afferenti all'IRST "Dino Amadori".

2.1.3 L'Assistente Virtuale e la Necessità di Dataset Dedicati

Un componente centrale nell'architettura della piattaforma CVL è lo sviluppo di un assistente virtuale basato su Large Language Models (LLMs), concepito per consentire a medici, oncologi e ricercatori di interrogare la base di conoscenza direttamente in linguaggio naturale (italiano o inglese). L'obiettivo primario è eliminare la barriera tecnica rappresentata dal linguaggio SPARQL: per un professionista sanitario, infatti, la formulazione di una query formale richiede non soltanto la padronanza di una sintassi rigida basata su grafi, ma anche la conoscenza approfondita e mnemonica della struttura dell'ontologia, dei nomi specifici dei predicati e dei vocabolari controllati adottati.

Per realizzare tale interfaccia, la semplice adozione di un modello linguistico generico di grande taglia (*off-the-shelf*) o l'affidamento a servizi commerciali via API esterne risultano strategie inapplicabili, per tre ragioni fondamentali:

1. *Mancanza di conoscenza ontologica specifica (Ontology Gap)*: Un LLM pre-addestrato su corpora generici non conosce le convenzioni di modellazione interne al Cancer Virtual Lab. Se interpellato su un dominio specialistico, il modello tende ad allucinare predicati plausibili nel linguaggio comune ma inesistenti nel grafo RDF di produzione, generando query sintatticamente corrette ma semanticamente vuote o ineseguibili.

2. *Vincoli di riservatezza e trattamento dei dati sanitari:* Le normative sulla protezione dei dati in ambito medico (quali il GDPR) e la presenza di dati clinici sensibili impongono l'adozione di modelli eseguiti interamente in locale. Ciò preclude l'uso di modelli cloud proprietari e richiede la specializzazione di architetture locali di dimensioni contenute, che necessitano di un addestramento mirato per compensare la minore capacità parametrica generale.
3. *Rigore sintattico ed esecuzione deterministica:* A differenza della generazione di testo naturale, la traduzione *Text-to-SPARQL* non ammette imprecisioni. Anche un singolo errore di formattazione, una parentesi non bilanciata o un identificativo errato rendono l'intera query ineseguibile dall'endpoint del triplestore.

La specializzazione di un modello linguistico locale per questo compito specifico richiede la tecnica del *Supervised Fine-Tuning* (SFT), la quale necessita a sua volta di un dataset di addestramento di alta qualità composto da coppie di domande in linguaggio naturale e query SPARQL corrispondenti.

Tuttavia, un dataset di questo tipo non esiste in partenza per l'ontologia di CVL. La sua costruzione manuale, affidata alla scrittura congiunta di esperti di dominio clinico e di ingegneri del Web Semantico, richiederebbe tempi inaccettabili e non garantirebbe la copertura sistematica delle combinazioni di vincoli possibili. La progettazione e l'implementazione di una pipeline automatizzata in grado di sintetizzare un dataset di addestramento accurato, vario e verificabile costituisce quindi il prerequisito fondamentale e la motivazione primaria del presente lavoro di tesi.

2.2 Knowledge Graph, RDF e Strumenti SPARQL

Per comprendere le scelte architettureali della piattaforma CVL e le sfide legate alla generazione delle query, è necessario formalizzare il modello di dati sottostanti. L'organizzazione delle informazioni cliniche non si basa su tabelle relazionali

tradizionali, ma sul paradigma dei Knowledge Graphs (KGs) e sugli standard del Web Semantico raccomandati dal W3C.

2.2.1 Dai Dati Strutturati ai Knowledge Graph

Un KG è una struttura dati orientata a grafo, progettata per rappresentare la conoscenza integrando entità, attributi e relazioni. A differenza delle basi di dati relazionali tradizionali, dove i dati sono vincolati a schemi rigidi suddivisi in tabelle e chiavi esterne, un Knowledge Graph esprime la conoscenza in modo flessibile ed esplicito:

- *I Nodi* rappresentano le entità concrete del dominio (es. un specifico paziente, una diagnosi, un farmaco);
- *Gli Archi Diretti* rappresentano le relazioni semantiche tipizzate che collegano un nodo all'altro (es. `ha_diagnosi`, `assume_farmaco`).

Questo approccio risulta particolarmente vantaggioso in ambito oncologico, dove la struttura dei dati è intrinsecamente complessa e soggetta a continua evoluzione, dove nuove proprietà o relazioni possono essere introdotte nel grafo senza dover ridefinire lo schema globale dell'intera base di dati.

2.2.2 Resource Description Framework (RDF)

Lo standard formale impiegato per concretizzare un Knowledge Graph è il Resource Description Framework (RDF). In RDF, qualsiasi informazione o asserzione viene scomposta nella sua unità elementare atomica chiamata *tripla*:

(Soggetto, Predicato, Oggetto)

Ciascun elemento della tripla svolge un ruolo preciso nella rete di conoscenza:

- *Soggetto*: identifica l'entità principale dell'asserzione mediante un Internationalized Resource Identifier (IRI), un indirizzo univoco a livello globale;
- *Predicato*: esprime la relazione specifica o la proprietà descritta, anch'esso identificato da un IRI ;

- *Oggetto*: rappresenta il punto di arrivo della relazione. Può essere a sua volta un IRI che punta a un'altra entità del grafo (dando origine a una struttura di nodi interconnessi) oppure un valore letterale (*Literal*) tipizzato (es. una stringa, un numero intero, una data ISO 8601 o un valore booleano).

L'insieme di tutte le triple forma il grafo RDF. Le regole formali, i vocabolari controllati e la gerarchia di classi ammesse all'interno del grafo costituiscono l'*ontologia* del sistema, ovvero lo schema concettuale del dominio clinico.

2.2.3 SPARQL

Per estrarre informazioni e interrogare un grafo RDF si utilizza SPARQL (*SPARQL Protocol and RDF Query Language*), il linguaggio di interrogazione standard per i dati del Web Semantico.

A differenza di SQL, che opera eseguendo operazioni di join tra tabelle, SPARQL funziona tramite il meccanismo del *graph pattern matching* (corrispondenza di schemi di grafo). La clausola **WHERE** di una query SPARQL definisce un insieme di "triple con incognite" (dette *triple pattern*), in cui alcuni elementi sono sostituiti da variabili identificate dal prefisso ? (es. ?patient, ?diagnosis). L'algoritmo di esecuzione confronta lo schema richiesto con il grafo RDF reale, individuando tutti i sotto-grafi che soddisfano simultaneamente le combinazioni di vincoli specificate.

Per raffinare la ricerca, SPARQL mette a disposizione clausole espressive quali:

- **FILTER**: applica vincoli logici, algebrici o temporali sui valori letterali (es. restringere la ricerca a pazienti nati entro un determinato intervallo di date);
- **VALUES**: limita i valori assegnabili a una variabile a un elenco discreto e prefissato;
- **OPTIONAL**: consente di estrarre attributi accessori senza compromettere la restituzione dei risultati principali qualora il dato non sia presente nel grafo per un determinato paziente.

2.2.4 Validazione Sintattica Determinista con RDFLib

Poiché le query SPARQL generate dalla pipeline sintetica dovranno servire da target per l'addestramento di un modello di linguaggio, la presenza di errori di sintassi comprometterebbe l'efficacia dell'apprendimento.

Per garantire la validità formale delle query senza dover sovraccaricare l'endpoint SPARQL di produzione con continue chiamate di rete, il sistema sfrutta la libreria Python `rdflib`. `rdflib` fornisce un parser sintattico deterministico che analizza l'albero sintattico astratto (*AST*) della query generata in locale. Qualora la query presenti parentesi non bilanciate, keyword errate o sintassi SPARQL 1.1 non conforme, il parser solleva un'eccezione, consentendo alla pipeline di scartare o rigenerare immediatamente l'esempio difettoso.

2.3 Architettura Software e Pattern

La progettazione di pipeline complesse per l'elaborazione di dati e la sintesi di corpora richiede l'adozione di modelli di programmazione e strumenti software in grado di garantire modularità, manutenibilità e rapidità di ispezione. In questa sezione vengono analizzati, a livello teorico e metodologico, i principali pattern di progettazione ad oggetti e i paradigmi di prototipazione rapida considerati nell'ambito del progetto.

2.3.1 Il Pattern Architetturale Mixin

Nello sviluppo di sistemi orientati agli oggetti, la gestione di processi complessi multi-stadio richiede una rigorosa separazione delle responsabilità per rispettare i principi SOLID, con particolare riferimento al principio di singola responsabilità (*Single Responsibility Principle*). Quando una singola classe accumula funzionalità eterogenee, quali la gestione della comunicazione di rete, il parsing dei dati, la logica di campionamento e le operazioni di I/O su disco, la struttura del codice tende a diventare monolitica, rigida e complessa da sottoporre a test.

Per risolvere tale criticità senza ricorrere ad architetture sproporzionatamente articolate, la programmazione orientata agli oggetti mette a disposizione il pattern *Mixin*. Un Mixin si definisce come una classe progettata non per rappresentare una

struttura di dati autonoma, bensì per fornire un insieme circoscritto e coerente di metodi a un'altra classe mediante l'impiego dell'ereditarietà multipla. A differenza dell'ereditarietà classica, la quale esprime una relazione di specializzazione fra tipi, il Mixin introduce una relazione di estensione funzionale.

Tale approccio comporta tre proprietà fondamentali sul piano teorico. In primo luogo, la classe Mixin non viene mai istanziata direttamente nel flusso di esecuzione. In secondo luogo, essa non mantiene uno stato interno complesso o un costruttore autonomo, bensì opera sui metodi e sugli attributi messi a disposizione dalla classe che la eredita. In terzo luogo, la classe principale, detta orchestratrice, può ereditare da più Mixin contemporaneamente, incorporandone le capacità all'interno di un'unica interfaccia unificata.

I vantaggi metodologici derivanti da questa architettura risiedono nella scomposizione del dominio applicativo in unità logiche isolate, nella garanzia di un'elevata coesione interna unita a un basso accoppiamento fra le parti e nella possibilità di riutilizzare i medesimi componenti in configurazioni differenti senza duplicazione di codice.

2.3.2 Prototipazione e Interfacce Reattive con Streamlit

Nelle fasi di calibrazione dei parametri di un algoritmo e di analisi esplorativa dei dati, l'interazione programmata da riga di comando presenta limiti di usabilità, poiché non offre un riscontro visivo immediato sullo stato dell'elaborazione o sulla struttura delle query prodotte.

Per ovviare a questo limite, la letteratura e la pratica ingegneristica orientata alla *Data Science* incoraggiano l'adozione di framework per la prototipazione rapida di interfacce web reattive, tra i quali figura **Streamlit**. A differenza delle architetture web tradizionali, che richiedono la separazione tra un servizio di backend e un'interfaccia frontend sviluppata con tecnologie dedicate, Streamlit si basa su un paradigma di esecuzione reattiva direttamente in linguaggio Python. Ogni interazione dell'utente con gli elementi di controllo dell'interfaccia determina un aggiornamento dello stato di sessione, consentendo di parametrizzare gli algoritmi, gestire eventi di interruzione e visualizzare i dati in tempo reale.

Poiché la piattaforma del Cancer Virtual Lab risulta interamente sviluppata mediante l'impiego di Streamlit, il primo stadio della pipeline di generazione è stato concepito per integrarsi nativamente all'interno dell'infrastruttura preesistente. Questa scelta architetturale ha permesso di estendere il sistema con un modulo di controllo avanzato, consentendo il monitoraggio interattivo dei processi di generazione e la sincronizzazione con l'ambiente operativo del progetto.

2.4 Lavori Correlati e Benchmark di Text-to-SPARQL

La costruzione di dataset di addestramento e benchmark per il compito di *Text-to-SPARQL* ha seguito un'evoluzione significativa negli ultimi anni, passando da corpora puramente sintetici a metodologie guidate da modelli linguistici. L'analisi dello stato dell'arte consente di classificare le tecniche di generazione in tre approcci principali, evidenziandone i limiti strutturali rispetto alle esigenze di un dominio specialistico come quello clinico-oncologico.

2.4.1 Approcci Basati su Template Rigidi e Corpora Sintetici

I primi approcci alla generazione di dataset per il Web Semantico si sono basati sull'impiego di template fissi associati a regole grammaticali formali. Benchmark pubblici di riferimento, quali LC-QuAD [TMDL17] e la sua successiva evoluzione LC-QuAD 2.0 [DBAL19], hanno adottato metodologie per la generazione su larga scala di coppie domanda-query mirate a basi di conoscenza generali come DBpedia e Wikidata.

In questo approccio, un insieme di modelli linguistici pre-strutturati viene riempito iterativamente con istanze estratte dal grafo. Sebbene tali dataset offrano volumi elevati di dati, presentano limiti strutturali insormontabili quando applicati a ontologie verticali: la rigidità dei template genera formulazioni linguistiche artificiali e ripetitive, incapaci di coprire la varietà lessicale e la complessità relazionale tipica del linguaggio umano specialistico-oncologico.

2.4.2 Sintesi Guidata da Regole e Grammatiche Formali

Per superare la rigidità dei template tradizionali, successivi lavori hanno esplorato tecniche di sintesi automatizzata basate sulla generazione orientata alla grammatica, come ad esempio la metodologia MK-SQuIT [SCKS20]. Questo approccio sfrutta grammatiche formali ed elementi di logica per assemblare in modo ricorsivo piccoli frammenti di frase con i rispettivi vincoli della query, costruendo così domande strutturalmente più articolate.

Sebbene tali sistemi migliorino la copertura delle combinazioni logiche, mantengono un forte legame con la struttura formale sottostante. Le domande prodotte tendono a rispecchiare fedelmente la struttura della query SPARQL anziché il modo in cui un operatore umano formulerebbe il quesito, risultando inadeguate ad addestrare modelli linguistici destinati a un'utenza clinica non tecnica.

2.4.3 Generazione Indotta da Large Language Model

Con l'avvento dei Large Language Model, la letteratura recente ha esplorato l'uso diretto di modelli generativi per la sintesi e l'arricchimento di dataset Text-to-SPARQL, come dimostrato da studi applicati in contesti aziendali e generali [LdCF25]. In questo paradigma, un LLM viene interpellato per parafrasare o generare direttamente domande a partire da query formali.

Ciononostante, l'affidamento incondizionato a un LLM per la sintesi end-to-end introduce criticità rilevanti:

- *Allucinazioni semantiche*: il modello tende a inventare relazioni o classi inesistenti nell'ontologia di riferimento;
- *Alterazione dei valori letterali*: identificativi, date e codici clinici critici vengono frequentemente modificati o omessi dal modello durante la generazione del testo;
- *Assenza di controlli strutturali*: l'assenza di un collegamento deterministico tra la query SPARQL e il prompt generato rende impossibile garantire la correttezza semantica senza un'accurata validazione esterna.

2.4.4 Motivazioni della Scelta Metodologica nel Contesto CVL

L'analisi dei lavori correlati evidenzia come nessuno degli approcci tradizionali sia direttamente applicabile al Cancer Virtual Lab. I benchmark generali non coprono l'ontologia oncologica IRST, i template rigidi falliscono sulla varietà linguistica e la generazione non vincolata tramite LLM espone al rischio inaccettabile di corruzione dei dati clinici.

Per tali ragioni, questo lavoro adotta una strategia ibrida e controllata. Un primo stadio impiega un generatore combinatorio basato sullo schema del grafo per garantire la correttezza formale delle query; a questo fa seguito un secondo stadio di augmentation linguistica guidato da LLM, rigorosamente vincolato da un meccanismo di mascheramento dei valori letterali.

2.5 Fine-tuning di Large Language Model

I Large Language Model (LLM) pre-addestrati su corpora generici possiedono una vasta conoscenza linguistica di base, ma spesso mostrano prestazioni insufficienti quando applicati a compiti specialistici che richiedono il rispetto di sintassi rigide o di ontologie di dominio, come la traduzione *Text-to-SPARQL* trattata in questa tesi. Per adattare un modello pre-addestrato a un compito specifico si ricorre alla tecnica del *fine-tuning*.

2.5.1 Adattamento supervisionato (SFT)

Il *Supervised Fine-Tuning* (SFT) consiste nel prosieguito dell'addestramento del modello su un dataset specifico composto da coppie di input e output desiderati. Durante questo processo, i pesi della rete neurale vengono aggiornati tramite retropropagazione dell'errore per minimizzare la discrepanza tra la sequenza generata e quella target, insegnando al modello la struttura sintattica del linguaggio di destinazione e la semantica dei predicati dell'ontologia. La scelta di *quanti* e *quali* parametri aggiornare durante questo processo distingue diverse strategie di fine-tuning, descritte di seguito.

2.5.2 Full Fine-Tuning

Nella sua forma più diretta, il fine-tuning aggiorna la totalità dei parametri del modello pre-addestrato, esattamente come avviene durante il pre-addestramento originale, ma su un dataset più piccolo e specifico. Questo approccio, detto *Full Fine-Tuning*, offre in linea di principio la massima capacità di adattamento, poiché nessun parametro resta vincolato al proprio valore originale.

Il costo computazionale è però proporzionalmente elevato. Oltre allo spazio necessario per i pesi del modello stesso, l'ottimizzatore deve mantenere in memoria stati aggiuntivi per ciascun parametro addestrabile (ad esempio, gli ottimizzatori della famiglia Adam richiedono due valori di momento per ogni peso), moltiplicando di fatto i requisiti di memoria rispetto alla sola dimensione del modello. Per modelli di grandi dimensioni, questo può rendere il Full Fine-Tuning impraticabile su hardware di classe consumer o con risorse di calcolo limitate.

2.5.3 Tecniche di adattamento efficiente (PEFT, LoRA e QLoRA)

Per ovviare a questo limite, sono state introdotte le tecniche di *Parameter-Efficient Fine-Tuning* (PEFT), che aggiornano solo una piccola porzione dei parametri del modello, o ne introducono di nuovi in quantità ridotta, lasciando la maggior parte dei pesi originali invariata.

Tra le metodologie PEFT più diffuse figura *Low-Rank Adaptation* (LoRA) [HSW⁺22] che, invece di modificare la matrice dei pesi originali $W_0 \in \mathbb{R}^{d \times k}$ del modello, approssima l'aggiornamento dei pesi ΔW tramite il prodotto di due matrici a rango ridotto $A \in \mathbb{R}^{r \times k}$ e $B \in \mathbb{R}^{d \times r}$, dove il rango $r \ll \min(d, k)$:

$$W = W_0 + \Delta W = W_0 + B \cdot A$$

Solo le matrici A e B , molto più piccole della matrice originale, vengono addestrate e mantenute in memoria con i relativi stati dell'ottimizzatore; W_0 resta congelata e viene caricata una sola volta. Questo approccio riduce drasticamente il numero di parametri addestrabili (spesso di oltre il 99%), abbattendo i requisiti di memoria rispetto al Full Fine-Tuning, a fronte di una capacità di adattamento inferiore in

linea di principio, poiché l'aggiornamento è vincolato a un sottospazio di rango basso.

L'evoluzione *Quantized LoRA* (QLoRA) estende questo principio quantizzando il modello di base congelato a una precisione di 4-bit (utilizzando il formato *NormalFloat4*), riducendo ulteriormente la memoria occupata da W_0 stessa e non solo dagli stati dell'ottimizzatore, consentendo di eseguire l'addestramento di LLM di grandi dimensioni su hardware di classe consumer o su singola GPU locale.

2.5.4 Funzione di perdita e formato delle istruzioni

Durante il fine-tuning per compiti di generazione di codice o query formali, i dati vengono tipicamente strutturati in formato istruzione (*Instruction Format*), dove il prompt di input definisce il contesto o il ruolo del sistema e l'output target rappresenta la sequenza da generare.

La funzione di perdita usata è la *Cross-Entropy Loss* calcolata sui token sequenziali. Per ottimizzare l'addestramento ed evitare che il modello spenda capacità computazionale per imparare a ripetere il prompt di input, si applica comunemente il mascheramento della loss sui token di input (*Data Collator for Completion Only*): la perdita viene calcolata ed elaborata esclusivamente sui token corrispondenti all'output target, forzando il modello a concentrare l'apprendimento unicamente sulla generazione della struttura formale corretta.

Capitolo 3

Analisi

3.1 Obiettivi

L'obiettivo primario del progetto è la creazione di una pipeline in grado di costruire un dataset annotato per il fine-tuning di un modello linguistico di dimensioni contenute, destinato alla traduzione di domande in linguaggio naturale in query SPARQL da eseguire sul knowledge graph del Cancer Virtual Lab. Da questo obiettivo generale derivano quattro obiettivi più specifici, che hanno guidato le scelte progettuali descritte nei capitoli successivi.

- O1** *Copertura selettiva ed estendibilità.* Il dataset deve coprire le parti dello schema del knowledge graph più rilevanti per l'applicazione, senza richiedere la copertura completa dello schema. Il sistema deve inoltre permettere di estendere progressivamente questa copertura aggiungendo nuove classi e relazioni, senza richiedere una riprogettazione del generatore.
- O2** *Varietà linguistica autentica.* Le domande generate devono presentare una varietà linguistica sufficiente a evitare formulazioni ripetitive o riconducibili a un unico template, mantenendo invariato il significato espresso dalla query sottostante.
- O3** *Tracciabilità completa.* Ogni domanda generata deve essere associata in modo univoco alla query SPARQL da cui deriva. L'associazione tra doman-

da e query deve essere quindi conservata esplicitamente, così da rendere verificabile ogni coppia utilizzata nel dataset.

- O4** *Volume di dati adeguato al fine-tuning.* Il dataset deve contenere un numero di esempi sufficiente per supportare il fine-tuning del modello linguistico considerato e deve garantire una copertura adeguata delle diverse tipologie di query previste.

3.2 Vincoli

Il progetto è stato condotto sotto un insieme di vincoli tecnici che ne hanno influenzato le scelte architetturali, descritte nel dettaglio nei capitoli successivi.

- V1** *Compatibilità con uno schema esistente e non modificabile.* Il sistema deve generare query conformi allo schema dell'ontologia del Cancer Virtual Lab, preesistente al progetto e non alterabile nel suo ambito. Nomi di classi, proprietà e relazioni sono vincoli esterni, non negoziabili in funzione di una generazione più semplice o di una copertura più ampia.
- V2** *Trattamento di dati clinici sensibili.* I valori coinvolti nelle query come identificativi di pazienti, codici di classificazione clinica o date sono dati clinici il cui significato deve rimanere verificabile e inalterato in ogni fase del processo. Questo vincolo esclude, inoltre, l'invio di tali dati a servizi esterni non controllati, imponendo l'esecuzione locale di qualunque componente del sistema che vi acceda.
- V3** *Capacità limitata del modello linguistico impiegato.* A causa del vincolo sull'esecuzione locale imposto da V2, il sistema potrà sfruttare solo modelli di dimensioni molto contenute, non potendo quindi fare affidamento sulla correttezza semantica spontanea dell'output prodotto, e richiede un meccanismo di validazione esplicito e indipendente dal modello stesso.

3.3 Requisiti funzionali

I requisiti funzionali descrivono cosa il sistema deve produrre, indipendentemente da come tale produzione venga realizzata.

- RF1** *Generazione di coppie prompt-query.* Il sistema deve generare coppie costituite da una query SPARQL sintatticamente valida e da un prompt in linguaggio naturale corrispondente, a partire dallo schema del knowledge graph.
- RF2** *Produzione di varianti multiple.* Il sistema deve produrre, per ciascuna coppia generata, un numero variabile di varianti parafrasate del medesimo prompt, preservando invariata la query associata.
- RF3** *Preservazione dei valori letterali.* Il sistema deve garantire, in ciascuna variante parafrasata, la presenza dei valori letterali presenti nella query, quali identificativi di pazienti, codici di classificazione clinica o date, nella loro forma esatta o in una forma alternativa verificabile.
- RF4** *Complessità variabile delle query.* Il sistema deve supportare la generazione di query di complessità variabile, dal caso di un singolo vincolo su una singola proprietà fino a query che attraversano più relazioni e combinano più vincoli sullo stesso nodo.

3.4 Requisiti non funzionali

I requisiti non funzionali riguardano invece proprietà trasversali del sistema, non direttamente osservabili nel singolo output prodotto.

- RNF1** *Sostenibilità computazionale.* Il sistema deve garantire un tasso di accettazione delle generazioni sufficientemente elevato da rendere il processo economicamente sostenibile in termini di tempo di calcolo.
- RNF2** *Resilienza a interruzioni.* A causa del vincolo sull'esecuzione locale imposto da V2, che comporta risorse di calcolo limitate, tempi di elaborazione prolungati e un maggior rischio di interruzioni, il sistema deve adottare meccanismi

che permettano di limitare la perdita di lavoro già svolto in caso di arresto imprevisto.

RNF3 *Robustezza semantica.* Il sistema deve limitare il rischio che la variazione linguistica modifichi, senza rilevarlo, il significato della domanda rispetto alla query sottostante, con particolare attenzione ai valori clinici controllati.

Nota *La robustezza semantica riguarda quindi la fedeltà del contenuto, mentre la tracciabilità di O3 riguarda la provenienza e l'associazione tra testo e query.*

Capitolo 4

Progettazione

In questo capitolo vengono illustrate le decisioni progettuali e le strategie architetturali adottate per soddisfare i requisiti e i vincoli definiti nella fase di analisi. La trattazione muove da una visione architetturale ad alto livello, incentrata sulla coordinazione tra i macro-componenti del sistema, per poi scendere nel dettaglio dei singoli sottosistemi, evidenziando le motivazioni ingegneristiche alla base di ciascuna scelta.

4.1 Analisi dei problemi

I requisiti e i vincoli descritti nel capitolo precedente si traducono in tre sotto-problemi progettuali distinti, la cui interazione determina la difficoltà complessiva del compito. La distinzione è utile perché consente di separare le difficoltà legate alla costruzione delle query da quelle introdotte dalla variazione linguistica e, infine, dalla necessità di verificare gli output prodotti.

P1 *Sotto-problema combinatorio.* Dato lo schema del knowledge graph, generare query SPARQL sintatticamente corrette che coprano una distribuzione rappresentativa dell'uso reale del sistema, senza produrre un numero di combinazioni non gestibile nei tempi o eccessivo per un corretto fine-tuning.

P2 *Sotto-problema linguistico.* Dato un prompt in forma template privo di variazione, generare un numero variabile di parafrasi che siano al tempo stesso

fluenti in linguaggio naturale e fedeli al contenuto semantico della query sottostante. A differenza di P1, qui la difficoltà non risiede nella dimensione dello spazio delle soluzioni possibili, ma nell'assenza di un criterio puramente formale per distinguere una parafrasi corretta da una scorretta. Una parafrasi può infatti essere grammaticalmente ineccepibile e superare ogni controllo meccanico applicabile, e tuttavia alterare il significato della domanda originale.

P3 *Sotto-problema di validazione.* Distinto da P1 e P2 ma dipendente da entrambi: verificare, per ciascuna variante generata, che la relazione con la query associata sia corretta e che il contenuto semantico non sia stato alterato durante la variazione linguistica. La tracciabilità richiesta da O3 è una proprietà verificabile della provenienza del dato; la robustezza semantica di RNF3 introduce invece una difficoltà ulteriore, poiché non esiste un criterio puramente formale che stabilisca con certezza l'equivalenza tra una formulazione in linguaggio naturale e la rappresentazione SPARQL sottostante. P3 è quindi il punto in cui la pipeline deve combinare controlli strutturali e semantici, ed è discusso nel dettaglio nel capitolo di valutazione.

I tre sotto-problemi vengono affrontati da componenti distinti della pipeline. P1 è affrontato attraverso il generatore parametrico di query, che consente di controllare lo spazio delle configurazioni esplorate. P2 è affrontato attraverso la fase di augmentation linguistica, progettata per produrre formulazioni diverse a partire dalla medesima query. P3 richiede invece una fase di validazione, che verifica gli output prodotti dalle fasi precedenti prima del loro inserimento nel dataset.

4.2 Design del generatore

Questa sezione descrive le decisioni progettuali alla base del primo stadio del sistema, responsabile della generazione strutturata delle coppie prompt-query.

4.2.1 La pipeline di generazione

Generare una singola coppia prompt-query valida a partire da un'entità dello schema richiede una sequenza di passaggi logicamente distinti. Risulta infatti necessario individuare le proprietà e le relazioni disponibili per l'entità selezionata, determinare i percorsi che consentono di raggiungerle all'interno del grafo, selezionare i vincoli da applicare nel rispetto dei limiti configurati, tradurre tali selezioni in una struttura di query, riempire la struttura con valori concreti, formulare il corrispondente testo in linguaggio naturale e, infine, verificare la correttezza formale del risultato ottenuto.

Accorpare tutte queste operazioni all'interno di un'unica procedura indifferenziata renderebbe la struttura rigida e complessa da sottoporre a test, poiché la modifica di un singolo aspetto, quale la logica di selezione dei valori, rischierebbe di ripercuotersi impropriamente sui componenti adibiti alle altre trasformazioni.

Per superare tale criticità, il generatore struttura il processo come una pipeline composta da fasi ben definite, in cui ciascun modulo riceve in ingresso il risultato del passaggio precedente e ne produce l'input per il successivo.

In fase di progettazione, questa scomposizione si riflette direttamente nell'adozione del pattern architetturale *Mixin*: a ciascuna fase concettuale della pipeline corrisponde un Mixin dedicato, responsabile di un'unica trasformazione isolata. Tale approccio rende ogni componente autonomamente verificabile e predispone una corrispondenza diretta tra il modello concettuale del generatore e la sua architettura software, descritta nel dettaglio in sezione 5.1.1.

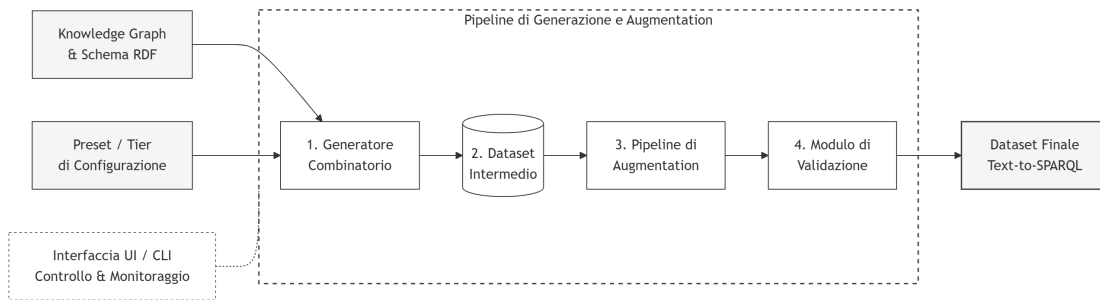


Figura 4.1: Architettura macro ad alto livello della pipeline dati: flusso sequenziale dal Knowledge Graph fino alla validazione del dataset finale.

4.2.2 Modello del dominio

Il modello di dominio di questo progetto non coincide con l'intero schema del Knowledge Graph del Cancer Virtual Lab, ma con un sottoinsieme di 45 classi fornito dall'IRST. La selezione delle classi non ha seguito un criterio puramente tecnico o esaustivo, quale l'inclusione indiscriminata di ogni nodo disponibile nel triplestore. Ma, al contrario, è stata strettamente guidata dagli obiettivi clinici del progetto quali la costruzione di un dataset focalizzato su query ad alta utilità operativa, ovvero capaci di rispecchiare i quesiti informativi realmente posti da medici e ricercatori nella pratica oncologica (quali l'individuazione di coorti di pazienti, il tracciamento dei protocolli terapeutici e l'analisi della risposta ai trattamenti). Questo sottoinsieme e la struttura relazionale che lo legano costituiscono un vincolo imposto dal grafo ricevuto; di conseguenza, le decisioni progettuali affrontate in questa sezione riguardano il modo in cui organizzare e pesare tali classi ai fini della generazione, e non la scelta di quali includere.

Le 45 classi sono organizzate in gruppi di priorità decrescente (figura 4.2) secondo un criterio di distanza strutturale dal nodo **Patient**. L'adozione di tale metrica risponde a un'osservazione sul dominio applicativo stesso: **Patient** costituisce la radice naturale del grafo clinico, coerentemente con la modalità con cui un operatore sanitario formula un quesito, ovvero a partire dall'entità paziente e non da una classe periferica (quale può essere un singolo biomarcatore). La distanza dalla radice offre un criterio oggettivo e calcolabile direttamente dalla topologia del grafo, evitando l'imposizione arbitraria di un giudizio soggettivo sulla rilevanza delle classi. Inoltre, tale misura rappresenta un'efficace approssimazione della complessità delle query, basti pensare che una classe raggiungibile con un solo passaggio relazionale genera tipicamente domande dirette, mentre una classe raggiungibile con due o tre passaggi richiede query che concatenano più relazioni, fornendo così un'indicazione sia sulla centralità clinica sia sulla difficoltà attesa del quesito.

L'assenza di un criterio di priorizzazione avrebbe comportato conseguenze dirette sul dataset prodotto, discusse in relazione a O4 nel capitolo di analisi. Trattando le 45 classi come equivalenti, il sistema avrebbe allocato un budget di generazione identico a ciascuna. Di conseguenza, classi concettualmente centrali quali

Patient, **Diagnosis** e **Therapy** avrebbero ottenuto una rappresentazione insufficiente rispetto alla frequenza d'uso reale; al contrario, classi periferiche o altamente granulari (come le singole categorie della classificazione TNM) avrebbero prodotto un numero di esempi ridondante. Un simile sbilanciamento avrebbe compromesso la capacità del modello addestrato di generalizzare in modo efficace sulle categorie di domande più frequenti.

La distanza dal nodo **Patient** non costituisce l'unico criterio di raggruppamento adottato. Un secondo schema, di natura tematica, organizza le medesime 45 classi per area clinica (demografia, stadiazione, trattamento, ecc.) ed è stato introdotto in una fase successiva, illustrata nel capitolo di implementazione. I due criteri risultano complementari: la distanza strutturale fornisce una progressione pulita di complessità per un primo passaggio di generazione, mentre il raggruppamento tematico ricombina le classi secondo relazioni di co-occorrenza differenti, consentendo a un secondo passaggio di produrre query multi-relazionali strutturalmente nuove.

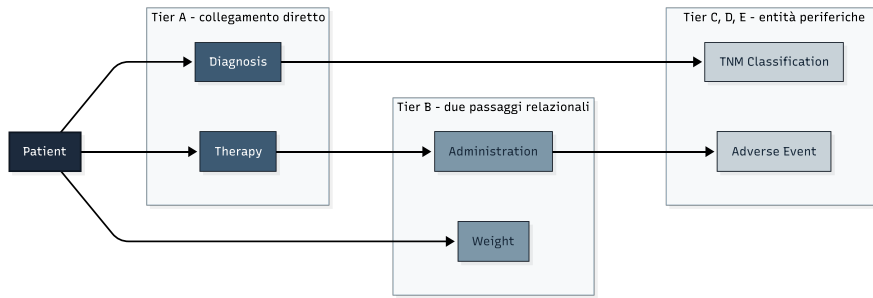


Figura 4.2: Modello di dominio semplificato, si noti Patient come nodo centrale

Il numero di record generati per ciascun gruppo decresce dal centro alla periferia del dominio, seguendo un andamento calibrato empiricamente per bilanciare l'equilibrio finale del dataset, piuttosto che una formula matematica definita a priori. Lo stesso principio di calibrazione empirica viene applicato al sistema dei quattro parametri descritto nella sezione seguente.

In sintesi, il dataset Text-to-SPARQL prodotto non è generato in modo uniforme sull'intero schema disponibile, ma rispetta una gerarchia di rilevanza clinica. Le classi incluse e i parametri di complessità applicati riflettono le tipologie di query ritenute prioritarie nel contesto del Cancer Virtual Lab, privilegiando la

pertinenza di dominio rispetto alla massimizzazione della copertura formale del grafo.

4.2.3 Sistema dei parametri

Il modello di dominio descritto nella sezione precedente definisce quali classi partecipano alla generazione e con quale peso relativo, ma non specifica in alcun modo la complessità delle singole query generabili su ciascuna di esse.

Il generatore espone quattro parametri indipendenti, ciascuno regolabile entro un intervallo minimo e massimo, la cui combinazione determina la complessità della query risultante:

- il numero di relazioni attraversate nel grafo a partire dal nodo radice;
- il numero di percorsi alternativi combinati nella medesima query tramite clausole di disgiunzione;
- il numero di proprietà vincolate su un medesimo nodo;
- il numero di valori ammessi per ciascun vincolo.

Disaccoppiare questi quattro parametri consente di modulare la struttura della query in modo mirato. In questo modo è possibile, ad esempio, aumentare la profondità delle relazioni sul grafo senza necessariamente incrementare il numero di filtri sui singoli nodi, evitando la generazione di query strutturalmente sbilanciate o non calibrate per lo schema di destinazione.

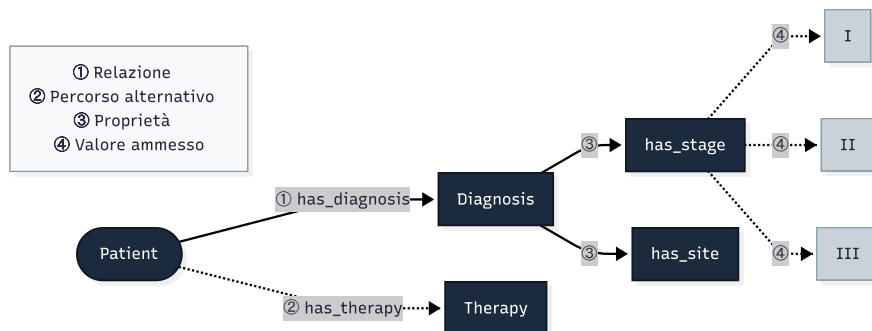


Figura 4.3: Modello esemplificativo dei parametri filtrabili

Sebbene l'impianto concettuale dei quattro parametri sia rimasto invariato nel corso del progetto, i valori numerici effettivamente assegnati a ciascuno di essi non sono stati definiti una sola volta, ma calibrati progressivamente attraverso diverse configurazioni successive, tutte vincolate da un limite superiore imposto dall'endpoint SPARQL stesso. Infatti query eccessivamente complesse, al crescere del numero di relazioni e proprietà vincolate simultaneamente, incorrevano in timeout o fallivano l'esecuzione contro il triplestore reale, rendendo necessario mantenere ciascun parametro entro un tetto massimo compatibile con le capacità dell'endpoint.

4.2.4 Meccanismo di estensione incrementale

Oltre a calibrare la complessità delle query, la struttura del sistema di parametri supporta l'estensione incrementale del dataset. Questa proprietà consente di integrare nuovi esempi in momenti successivi senza dover rielaborare le porzioni già generate e garantendo l'assenza di sovrapposizioni con il corpus esistente.

Il meccanismo si basa sulla definizione di regioni disgiunte all'interno dello spazio parametrico. Poiché ciascun parametro (profondità, disgiunzioni, proprietà e valori) è vincolato da un intervallo minimo e massimo, è possibile configurare un nuovo *preset* di generazione impostando intervalli distinti da quelli usati in precedenza su almeno una dimensione. Così facendo, il nuovo ciclo di generazione esplora esclusivamente combinazioni strutturali mai campionate prima.

In fase di progettazione, questa separazione garantisce l'estensibilità del sistema a due livelli:

- **Estensione parametrica:** permette di ampliare il volume di dati definendo nuovi file di configurazione (*tier*) esterni, senza modificare il codice sorgente del generatore;
- **Verifica dei duplicati:** l'assenza di sovrapposizioni garantita a livello teorico dalle configurazioni viene ulteriormente verificata durante la generazione, scartando eventuali query identiche prodotte per coincidenza stocastica.

L'aggiunta di nuove porzioni al dataset non richiede quindi una riprogettazione dell'architettura, ma la semplice definizione di nuovi file di configurazione i cui intervalli siano scelti in modo complementare rispetto a quelli già esplorati.

4.3 Design dell'augmentation

Questa sezione descrive le decisioni progettuali alla base del secondo stadio del sistema, responsabile della trasformazione delle coppie prompt-query prodotte dal generatore in più varianti linguistiche naturali, secondo il compromesso tra varietà e fedeltà semantica richiesto da O2 e la robustezza semantica richiesta da RNF3.

4.3.1 Mascheramento dei valori letterali e strategie di preservazione

Il secondo stadio della pipeline affronta la sfida di bilanciare due requisiti contrastanti individuati nella fase di analisi; da un lato massimizzare la varietà linguistica dei prompt per evitare formulazioni rigide (O2, RF2), dall'altro garantire la rigida fedeltà semantica e la preservazione dei valori clinici rispetto alla query SPARQL associata (RF3, RNF3).

Lasciare che un modello linguistico riscriva liberamente l'intero testo, valori compresi, esporrebbe il sistema ai rischi già identificati nel capitolo 3, quali l'alterazione o la perdita silenziosa di identificativi, codici di classificazione o date.

Per risolvere questo problema, l'architettura adotta un pattern di mascheramento deterministico che disaccoppia la struttura sintattica dai dati puntuali. Prima di interpellare il modello, ogni valore letterale referenziato nella query viene sostituito da un token segnaposto opaco (es. [VAL_A1B2]). Il modello opera così esclusivamente sulla componente sintattica e relazionale della frase, privo di accesso ai dati reali, i quali vengono ripristinati solo al termine della generazione. In questo modo si garantisce per costruzione che la riformulazione non possa modificare i valori target.

Per superare i limiti espressivi e computazionali del mascheramento rigido, il design del modulo integra due strategie specifiche:

- **Traduzione proposizionale dei vincoli booleani:** Trattare i valori booleani come semplici stringhe letterali (es. `"true"` o `"false"`) produrrebbe prompt mascherati innaturali e sintatticamente rigidi. In fase di design si è stabilito di convertire i vincoli booleani, prima del mascheramento, in forme proposizionali naturali (es. trasformando una condizione su un evento avverso nelle preposizioni «con» o «senza» seguite dal termine clinico). Questa scelta risolve il problema della naturalezza espressiva senza delegare al modello la gestione del valore di verità.
- **Decomposizione in porzioni ad alta densità (Chunking):** All'aumentare del numero di segnaposto in un singolo prompt, la capacità del modello di conservare ed elaborare tutti i token opachi degrada, aumentando il tasso di allucinazioni o l'omissione involontaria di segnaposto. Per mitigare questo limite, il sistema applica un principio di decomposizione: i prompt con un numero di segnaposto superiore a una soglia prestabilita vengono suddivisi in porzioni logiche indipendenti (*chunk*), ciascuna contenente un budget limitato di token. Ogni porzione viene parafrasata separatamente e ricomposta nel testo finale, garantendo un tasso di accettazione elevato anche per query ad alta complessità combinatoria.

4.3.2 Generazione iterativa e validazione

Il mascheramento garantisce che i valori letterali, se presenti nell'output, siano corretti; non garantisce però che il modello produca sempre un output valido al primo tentativo, né che varianti successive dello stesso record non finiscano per ripetersi tra loro, vanificando l'obiettivo di varietà linguistica di O2.

Il sistema affronta questo problema attraverso un ciclo iterativo di generazione e verifica. Ogni candidato prodotto dal modello viene sottoposto a una fase di validazione che considera diversi criteri.

- **Preservazione dei segnaposto.** Il candidato viene rifiutato se anche un solo segnaposto è stato omesso, modificato o sostituito.

- **Assenza di segnaposto non previsti.** Viene verificato che il testo non contenga sequenze che imitano la sintassi di un segnaposto senza corrispondere a uno dei segnaposto effettivamente presenti nel testo originale.
- **Controllo della similarità.** Il candidato viene confrontato con le varianti già accettate per lo stesso record. Se risulta eccessivamente simile a una di esse, viene rifiutato. La soglia di similarità viene resa progressivamente più permissiva al diminuire della lunghezza del testo, così da evitare che formulazioni brevi vengano considerate erroneamente come duplicati.

In caso di rifiuto, il sistema non si limita a ripetere la generazione. Il candidato scartato e il relativo motivo vengono forniti al modello come contesto aggiuntivo per il tentativo successivo, in modo da ridurre la probabilità che venga ripetuto lo stesso errore. A ogni nuovo tentativo viene inoltre incrementata progressivamente la temperatura di campionamento, favorendo la produzione di una formulazione diversa da quella precedente.

Se anche l'ultimo tentativo disponibile viene rifiutato, il sistema non forza comunque l'inserimento di una nuova variante. Viene invece mantenuto il testo originale non modificato e viene registrato esplicitamente il ricorso a questo comportamento di ripiego. In questo modo si evita di accettare silenziosamente una formulazione che violi i vincoli definiti da RF3 o RNF3.

Inoltre, il budget di varianti target assegnato a ciascun record non viene definito in modo uniforme, ma calcolato dinamicamente tramite una funzione proporzionale alla densità di valori mascherati presenti nel prompt. Un testo caratterizzato da un numero ridotto di vincoli presenta uno spazio di variabilità linguistica ristretto; imporre una quota fissa di varianti a prompt semplici avrebbe generato un'elevata frequenza di rigetti per eccessiva similarità lessicale, saturando inutilmente il ciclo di tentativi e degradando l'efficienza complessiva del processo (RNF1). L'adozione di una soglia adattiva garantisce quindi che la pressione generativa sia concentrata sui record ad alta complessità combinatoria, massimizzando la varietà utile ed evitando cicli di retry ridondanti sui casi più semplici.

4.3.3 Varietà stilistica

Il solo mascheramento non garantisce, di per sé, la varietà linguistica richiesta da O2, anzi un modello lasciato libero di riformulare tenderebbe a convergere su un piccolo numero di strutture sintattiche ricorrenti. Il sistema affronta questo problema definendo un insieme di dieci registri stilistici predefiniti (ad esempio una formulazione tecnica in stile interrogazione a database, una clinico-formale, una colloquiale, una statistica orientata al conteggio), esplicitamente descritti al modello a ogni chiamata.

Stile	Formulazione della domanda
Tecnico/SQL	Estrai il conteggio totale dei record nella tabella pazienti che presentano la codifica 'Mammella femminile' e una data di nascita compresa nel range 01/01/1980 - 07/08/1995.
Clinico Formale	Quanti soggetti di sesso femminile con diagnosi oncologica alla mammella risultano nati nel periodo compreso tra l'inizio del 1980 e il 7 agosto 1995?
Basato sull'età	Fornisci il numero di pazienti con diagnosi 'Mammella femminile' che hanno attualmente un'età compresa tra i 30 e i 46 anni.
Gestionale/Reporting	Rapporto volumetrico: quante unità afferiscono alla patologia 'Mammella femminile' limitatamente alla coorte di nascita 1980-1995 (fino al 7 agosto)?
Linguaggio Naturale	Quante pazienti abbiamo in archivio che sono nate tra l'80 e metà agosto '95 e che soffrono di problemi alla mammella femminile?

Tabella 4.1: Esempio di variazione stilistica applicata a 5 registri rappresentativi a partire dal medesimo prompt mascherato.

Gli stili non vengono assegnati alle varianti in modo casuale, ma percorsi in sequenza ciclica, in modo che la distribuzione degli stili sulle varianti generate risulti uniforme anziché soggetta alla variabilità di un'estrazione casuale. Per ciascuno stile, il sistema mantiene inoltre una memoria a breve termine delle formulazioni e delle parole di apertura usate più di recente, così rifiutando eventuali nuovi candidati che riproponessero un'apertura già usata di recente per lo stesso stile o una riformulazione troppo simile a livello di intero testo, estendendo così il controllo di

varietà anche alla struttura delle prime parole della frase, non solo al suo contenuto complessivo.

Infine, per ciascun record il sistema conserva anche una formulazione di controllo generata senza mascheramento, ottenuta inviando l'intero testo al modello in un'unica richiesta diretta. Questa modalità non rappresenta un percorso di produzione alternativo, bensì un punto di confronto semplice (baseline) introdotto per misurare i reali benefici del mascheramento sulla preservazione dei dati e sulla qualità del testo. La valutazione comparativa tra i due approcci è trattata nella sezione 6.1.2.

Capitolo 5

Implementazione

In questo capitolo viene descritto come sono state implementate le varie parti del contributo di questa tesi, in maniera coerente con la progettazione svolta nel capitolo 4. La trattazione segue l'ordine cronologico in cui i due stadi del sistema sono stati sviluppati.

5.1 Implementazione del generatore

5.1.1 Architettura a mixin

La pipeline di generazione descritta nella sezione 4.2.1 è realizzata concretamente dalla classe `BulkDatasetGenerator`, la quale eredita mediante ereditarietà multipla da sei *mixin*, uno per ciascuna fase del processo.

In coerenza con quanto introdotto nella sezione 2.3, questa scelta architetturale applica il principio di singola responsabilità, garantendo una corrispondenza diretta tra le fasi concettuali e i moduli software. Il sistema espone così un'unica interfaccia orchestratrice per l'intero ciclo di elaborazione, consentendo di modificare o estendere la logica di una specifica fase intervenendo esclusivamente sul mixin dedicato, senza impattare sugli altri componenti.

Listing 5.1: Composizione della classe principale del generatore tramite mixin.

```
1 class BulkDatasetGenerator(  
2     FilterApiMixin,  
3     DiscoveryMixin,
```

```
4     SkeletonMixin,  
5     PlanningMixin,  
6     ProgressMixin,  
7     OutputMixin,  
8 ):  
9     # ...
```

Ciascun mixin realizza una o più delle fasi introdotte a livello di design, incapsulando le chiamate di rete o coordinando la logica di più fasi quando queste sono strettamente collegate tra loro:

- **FilterApiMixin** incapsula le chiamate di rete verso l'API interna di CVL;
- **DiscoveryMixin** costruisce la mappa del grafo dello schema;
- **SkeletonMixin** costruisce lo scheletro di una singola query;
- **PlanningMixin** determina quali combinazioni di proprietà devono essere campionate;
- **OutputMixin** esegue i controlli di qualità finali e gestisce la scrittura dei risultati su disco;
- **ProgressMixin** gestisce gli aspetti legati all'interazione con l'interfaccia Streamlit, tra cui l'aggiornamento della barra di avanzamento e la possibilità di interrompere l'esecuzione.

Le responsabilità non sono quindi indipendenti, ma cooperano secondo il flusso della generazione, ad eccezione di **ProgressMixin**, che costituisce invece una responsabilità trasversale, poiché può intervenire durante l'esecuzione delle diverse fasi senza rappresentare una fase autonoma della pipeline.

La separazione è particolarmente evidente nella gestione dell'interruzione della generazione (gestita proprio da **ProgressMixin**). Il metodo `_is_aborted()`, riportato nel listato 5.2, viene richiamato periodicamente durante la generazione per verificare se l'esecuzione debba essere interrotta.

Listing 5.2: Controllo dell'interruzione durante la generazione.

```
1 def _is_aborted(self) -> bool:  
2     if self._abort_event.is_set():
```

```
3         return True
4     if not self.config.get("abortable", True):
5         return False
6     if self._can_paint() and bool(
7         st.session_state.get("abort_generation", False)
8     ):
9         self._abort_event.set()
10    return self._abort_event.is_set()
```

Il controllo appena citato combina due meccanismi:

- l'evento di sincronizzazione interno `_abort_event`, che rappresenta lo stato di interruzione del run
- lo stato dell'interfaccia Streamlit, dal quale viene letto il valore di `session_state["abort_generat`

Quando l'utente richiede l'interruzione, il metodo imposta l'evento interno; le chiamate successive possono quindi rilevare direttamente tale stato senza dover interrogare nuovamente l'interfaccia.

Il controllo dello stato di Streamlit viene eseguito soltanto quando l'esecuzione è configurata come interrompibile, tramite `config["abortable"]`, e quando il contesto consente di interagire con l'interfaccia. In questo modo la stessa implementazione può essere utilizzata anche nei contesti che non espongono un controllo di interruzione, come le esecuzioni da riga di comando o un pannello di generazione batch.

Le costanti di classe di `BulkDatasetGenerator`, riportate nel listato 5.3, definiscono alcuni parametri di funzionamento condivisi dalle diverse fasi della generazione.

Listing 5.3: Costanti di classe utilizzate dal generatore.

```
1 class BulkDatasetGenerator(
2     # parameters
3 ):
4     BUFFER_SIZE = 50
5     MAX_DISCOVERY_WORKERS = 8
6     GENERATION_WORKERS = 4
7     DISCARD_SAMPLES = 500
8     VALUE_REROLLS = 4
9     MAX_STORED_VALUES = 5000
10    SAMPLING_GIVEUP_STREAK = 250
```

In particolare, tali parametri controllano la dimensione del buffer di scrittura, il grado di parallelismo utilizzato durante la scoperta e la generazione, il numero massimo di valori conservati per proprietà e la soglia di tentativi consecutivi senza successo oltre la quale il campionamento viene interrotto. A questi si aggiungono i parametri relativi alla resilienza delle chiamate di rete, costituiti da un timeout di base di 15 secondi, fino a due ripetizioni in caso di fallimento e un secondo di attesa tra un tentativo e l'altro (NON presenti nel listato 5.3).

5.1.2 Costruzione e indicizzazione della mappa del grafo

Per evitare che l'esplorazione dello schema sia vincolata dai tempi di latenza dell'I/O di rete, la fase di avvio esegue una scansione preliminare dell'ontologia tramite il modulo `DiscoveryMixin`. Il risultato viene strutturato in un dizionario Python in memoria (`graph_map`), che agisce da cache offline locale per l'intera esecuzione.

Come mostrato nel listato 5.4, l'implementazione organizza i dati mediante due tabelle hash annidate, create separatamente per le proprietà filtrabili e per le relazioni uscenti di ciascuna classe.

Listing 5.4: Struttura della mappa del grafo memorizzata in `graph_map`.

```
1 graph_map = {
2     "properties": {
3         "Patient": [
4             {
5                 "predicate": {"value": "has_gender", "label": "gender"},
6                 "datatype": "string",
7                 "n_values": 2,
8                 "values": [
9                     {"value": "F", "label": "Female"},
10                    {"value": "M", "label": "Male"}
11                ]
12            }
13        ]
14    },
15    "relations": {
16        "Patient": [
17            {
18                "predicate": {"value": "has_diagnosis", "label": "diagnosis"},
19                "target": {"value": "Diagnosis", "label": "Diagnosis"}
20            }
21        ]
22    }
```

```
22     }  
23 }
```

La scelta di questa struttura dati e le relative modalità di popolamento rispondono a tre precise esigenze implementative:

- **Accesso in tempo costante $\mathcal{O}(1)$ durante la DFS:** Durante la ricerca in profondità dei percorsi (sezione 5.1.3), l'indicizzazione per chiave di entità consente l'estrazione immediata di proprietà e relazioni, evitando la scansione sequenziale di una lista globale ($\mathcal{O}(N)$).
- **Gestione della memoria (MAX_STORED_VALUES):** Per i campi ad alta cardinalità (es. misurazioni continue), il `DiscoveryMixin` applica un limite rigido alla lista `values`, memorizzando unicamente un campionario rappresentativo per la successiva fase di iniezione.
- **Parallelizzazione dell'I/O (MAX_DISCOVERY_WORKERS):** Poiché le chiamate REST per la scoperta delle singole entità sono operazioni indipendenti, il recupero iniziale può essere velocizzato sfruttando un pool di thread tramite `concurrent.futures.ThreadPoolExecutor`¹.

L'uso di una mappa statica, però, introduce un'approssimazione di dominio: Il sistema infatti assume che le proprietà di una classe dipendano unicamente dal suo tipo e non dalla sequenza di salti relazionali con cui viene raggiunta, al contrario rispetto l'API di CVL, la quale adotta un comportamento dinamico che mostra o nasconde determinati vincoli in base allo storico della selezione.

Questa scelta progettuale è stata deliberatamente accettata a seguito di un'analisi di fattibilità:

- **Abbattimento della latenza di rete:** Senza la mappa in memoria, ogni nodo della DFS richiederebbe un'interrogazione HTTP sincrona. Nelle strutture ad alta complessità combinatoria, il volume di chiamate aumenterebbe in modo moltiplicativo, portando i tempi di generazione da pochi minuti a diverse ore con un alto rischio di timeout.

¹<https://docs.python.org/3/library/concurrent.futures.html#threadpoolexecutor>

- **Impatto marginale sulla copertura (O1):** La mancata tracciabilità del contesto dinamico priva il sistema solo di predicati altamente specialistici legati a combinazioni rare, preservando appieno la rappresentazione delle query più frequenti e rilevanti per il dominio clinico.

5.1.3 Costruzione delle route

Con la mappa disponibile, per ciascuna entità radice richiesta, si esplora tramite una visita in profondità (DFS) ogni combinazione raggiungibile di salti relazionali fino a un limite di profondità configurato.

Listing 5.5: Visita in profondità della mappa del grafo per la costruzione delle route disponibili a partire da un'entità radice.

```
1 def dfs(type_value, chain, relations_used, visited, ancestor_terminals):
2     terminals = [
3         {"predicate": p["predicate"], "chain_prefix": list(chain), ...}
4         for p in properties.get(type_value, [])
5     ]
6
7     if terminals:
8         routes.append({
9             "chain": list(chain),
10            "terminals": ancestor_terminals + [{"**t, "is_final_entity": True} for
11                t in terminals],
12        })
13
14    if relations_used >= max_relations:
15        return
16
17    inherited = ancestor_terminals + [{"**t, "is_final_entity": False} for t in
18        terminals]
19
20    for rel in relations.get(type_value, []):
21        target_value = (rel.get("target") or {}).get("value")
22
23        if not target_value or target_value in visited:
24            continue
25
26        dfs(
27            target_value,
28            [*chain, {"predicate": rel["predicate"], "reference_to": rel.get("
29                target")}],
30            relations_used + 1,
31            visited | {target_value},
32            inherited
```

Ogni volta che la visita raggiunge un nodo con proprietà filtrabili, viene registrata una *route*, ovvero l'insieme di:

- la sequenza di salti relazionali per raggiungerlo (la *chain*);
- l'insieme delle proprietà disponibili in quel punto (i *terminal*), che include sia quelle del nodo corrente sia quelle ereditate dai nodi attraversati in precedenza.

La distinzione dei terminal in proprietà ereditate e non (tramite variabile booleana) garantisce, nel passo successivo di campionamento, che ogni combinazione di proprietà scelta includa almeno un vincolo sul nodo in cui la route si ferma davvero, evitando di costruire query che vincolano solo nodi di passaggio.

Sebbene la visita che le genera sia intrinsecamente ad albero, l'insieme delle route risultanti viene appiattito in una singola lista di record indipendenti per facilitare la fase di campionamento (sezione 5.1.4), la quale deve poter scorrere ed estrarre a caso qualunque route tra tutte quelle disponibili, con accesso diretto e costante a ciascuna, senza dover ripercorrere un albero ogni volta per ricostruire chain e terminal di un nodo scelto casualmente al suo interno.

5.1.4 Pianificazione: stima combinatoria e campionamento

Prima di generare i record, il sistema calcola quante coppie prompt-query distinte siano effettivamente raggiungibili con le route disponibili e i vincoli configurati, in modo da non richiedere più record di quanti lo spazio combinatorio possa realmente offrire; solo dopo, campiona a caso tra le combinazioni così stimate per costruire ogni singolo record.

Stima combinatoria

Sia r una route, T_r l'insieme dei suoi terminal e $T_r^f \subseteq T_r$ i terminal finali (quelli del nodo dove la route si ferma davvero). Sia $v(t)$ il numero di combinazioni di valori di un terminal, e $\mathcal{C}_k(T_r)$ le combinazioni di k terminal di T_r (nel senso standard,

$\binom{|T_r|}{k}$ possibilità) che includono almeno un terminal di T_r^f . Con $p_{\min}$, $p_{\max}$ i limiti *min_properties*/*max_properties* configurati, il peso di r è

$$w(r) = \begin{cases} 0 & \text{se } T_r^f = \emptyset \\ \sum_{k=p_{\min}}^{\min(p_{\max}, |T_r|)} \sum_{c \in \mathcal{C}_k(T_r)} \prod_{t \in c} v(t) & \text{altrimenti.} \end{cases} \quad (5.1)$$

Un record spesso combina più route insieme. Sia V l'insieme delle route con peso non nullo, e $\text{indip}(S)$ vera quando nessuna coppia di route in S ha una chain prefisso dell'altra (sezione 5.1.3). Con $s_{\min}$, $s_{\max}$ i limiti *min_paths*/*max_paths* configurati, il tetto teorico dell'entità è

$$N = \sum_{s=s_{\min}}^{s_{\max}} \sum_{\substack{S \subseteq V \\ |S|=s \\ \text{indip}(S)}} \prod_{r \in S} w(r). \quad (5.2)$$

È da sottolineare che N è di fatto una sovrastima, in quanto route indipendenti che condividono un prefisso condividono anche parte dei terminal, mentre il prodotto dei pesi utilizzato nella nostra formula le tratta come scelte separate. Per l'uso che se ne fa, cioè riconoscere quando lo spazio delle combinazioni è troppo piccolo rispetto ai record richiesti, un limite superiore basta, perciò possiamo evitare di complicare ulteriormente il calcolo. Proprio per evitare un'ulteriore inefficienza, l'algoritmo imposta una soglia di 500 000 combinazioni da esaminare, dopo la quale il conteggio esatto viene abbandonato e il campionamento procede senza un tetto noto in anticipo.

Campionamento

Per ogni record da generare, il numero di route da combinare viene scelto a caso entro i limiti configurati. Le route disponibili vengono poi estratte da un elenco mescolato in ordine casuale, scartando quelle che risultano *in conflitto* con una già scelta, cioè la cui chain è un prefisso di un'altra già selezionata, il che indicherebbe due vincoli ridondanti sullo stesso tratto di percorso. Il campionamento continua finché non si raggiunge il numero di record stimato dal tetto teorico (equazio-

ne (5.2)), oppure finché 250 tentativi consecutivi (`SAMPLING.GIVEUP_STREAK`) non producono nulla di nuovo, segno che lo spazio raggiungibile è stato esaurito.

Listing 5.6: Campionamento casuale di un insieme di route indipendenti tra loro per un singolo record.

```
1 def sample_independent_combo(target_p):
2     pool = list(valid)
3     rng.shuffle(pool)
4
5     chosen, chosen_keys = [], []
6
7     for r in pool:
8         if len(chosen) >= target_p:
9             break
10
11         key = chain_key_of[id(r)]
12
13         if not routes_mod.conflicts(chosen_keys, key):
14             chosen.append(r)
15             chosen_keys.append(key)
16
17     return tuple(chosen)
```

Questa modalità di campionamento casuale (`_run.sampled`) non è l'unica prevista dal codice, esiste infatti anche una modalità a forza bruta (`_run.brute_force`) che enumera esaustivamente ogni combinazione possibile di route e proprietà. Quest'ultima modalità risulta utile quando l'obiettivo è ottenere una copertura completa dello spazio combinatorio; tuttavia non è stata utilizzata nei run di produzione, ma è stata comunque impiegata in fase di sviluppo per individuare eventuali errori di generazione, proprio grazie alla sua capacità di esplorare lo spazio delle combinazioni senza lasciarne fuori nessuna. Nella casistica affrontata da questa tesi però il dataset generato è destinato al fine-tuning, per il quale non è necessario enumerare tutte le combinazioni possibili, anzi è preferibile costruire un insieme sufficientemente ampio e vario di esempi rappresentativi, evitando al contempo di produrre una quantità di dati eccessiva rispetto allo scopo.

5.1.5 Esecuzione sequenziale e parallela

Una volta pianificata una combinazione di route e proprietà da trasformare in un record, ha inizio l'intero processo di costruzione del record vero e proprio, il

quale comporta diverse chiamate di rete verso l'API di CVL. Questo processo può essere eseguito in sequenza, una combinazione alla volta, oppure in parallelo su più thread, ciascuno impegnato a costruire un record diverso in contemporanea agli altri, a seconda della configurazione.

Listing 5.7: Scelta tra esecuzione sequenziale e parallela in base al numero di worker configurati.

```
1 workers = max(1, int(self.config.get("num_workers") or self.GENERATION_WORKERS))
2
3 if workers <= 1:
4     self._run_jobs_sequential(jobs_iter, total_pairs, p, target, max_no_new)
5     return
```

Se il numero di worker configurato è maggiore di uno, le combinazioni campionate vengono sottomesse a un pool di thread, fino a `GENERATION_WORKERS` worker attivi contemporaneamente (di default 4), ciascuno impegnato nella costruzione di un record indipendente. Il sistema non sottomette tuttavia tutte le combinazioni disponibili in un'unica operazione, poiché ciò potrebbe determinare l'accumulo in coda di un numero molto elevato di job in attesa. Viene invece mantenuta una finestra di lavori sottomessi ma non ancora completati, di dimensione pari al doppio del numero di worker attivi; quando la generazione di un record termina e il risultato viene salvato, il relativo spazio viene immediatamente riutilizzato per sottomettere una nuova combinazione.

Questa strategia consente, in condizioni di parallelismo effettivo, di mantenere i worker occupati riducendo i tempi morti, senza introdurre una quantità di lavoro in coda eccessiva rispetto alla capacità di elaborazione disponibile. Il meccanismo è stato predisposto principalmente in vista di future esecuzioni su larga scala, nelle quali il parallelismo potrebbe ridurre significativamente il tempo complessivo di generazione.

Durante la produzione del dataset trattato, il numero di worker è stato invece impostato esplicitamente a 1 (listato 5.8) eseguendo così la generazione in modalità single-threaded. La scelta è dettata dal collo di bottiglia rappresentato dalle chiamate all'API, le cui richieste concorrenti non avrebbero ridotto i tempi in proporzione, anzi avrebbero rischiato di comprometterne stabilità e prestazioni. Il parallelismo è rimasto invece attivo nella scoperta della mappa (sezione 5.1.2),

dove il carico delle operazioni si presta meglio all'esecuzione concorrente.

Proprio perché il parallelismo può comunque essere attivato lo stato condiviso tra i thread non è protetto da un unico lock globale ma da quattro lock distinti, ciascuno a guardia di una porzione di stato diversa e indipendente dalle altre:

- un lock protegge gli insiemi usati per riconoscere i duplicati
- un altro la cache dello scheletro (sezione 5.1.6)
- un altro ancora le statistiche interne sulle chiamate effettuate
- l'ultimo il registro dei job attualmente in esecuzione (sezione 5.1.9)

Tenerli separati, invece di usarne uno solo per tutto, evita che un thread resti bloccato in attesa di una struttura dati che non gli serve, solo perché un altro thread ne sta aggiornando una diversa nello stesso istante.

5.1.6 Costruzione dello scheletro: percorso online e offline

Ogni combinazione campionata deve essere tradotta in una struttura dati intermedia, chiamata *scheletro*, che rappresenta i nodi del grafo coinvolti nella query, prima ancora che i valori concreti vengano scelti. Il sistema offre due percorsi alternativi per costruirla, insieme ad altre opzioni di comportamento, tutte raccolte in un'unica configurazione condivisa da entrambe le liste di tier.

Le opzioni condivise

Listing 5.8: Configurazione condivisa usata nei run di produzione, per entrambe le liste di tier.

```
1 COMMON_CONFIG = {  
2     "num_workers": 1,  
3     "offline_history": True,  
4     "include_value_spans": True,  
5     "skip_navigation_in_prompt": False,  
6     "validate_sparql": True,  
7 }
```

Ciascuna di queste cinque opzioni governa un aspetto distinto del comportamento del generatore.

- `num_workers` a 1 è la scelta, già motivata in sezione 5.1.5, di eseguire la generazione in sequenza anziché in parallelo.
- `include_value_spans` controlla se il generatore debba tracciare le posizioni dei valori letterali nel prompt (sezione 5.1.8, dove è ripreso in dettaglio).
- `skip_navigation_in_prompt`, se attivato ometterebbe dal testo del prompt le clausole di sola navigazione, rendendolo più corto ma meno esplicito sul percorso nel grafo; è stato tenuto disattivato in produzione per non perdere quell'informazione.
- `validate_sparql` controlla se la query SPARQL restituita dall'API debba passare un controllo di sintassi prima di essere accettata; sarà ripreso più avanti insieme agli altri controlli di qualità (sezione 5.1.9).
- `offline_history` decide se lo scheletro venga ricostruito in locale oppure costruito camminando dal vivo dentro l'API; riguarda direttamente le due sottosezioni seguenti.

Il percorso online

Quando `offline_history` è disattivato, lo scheletro viene costruito camminando passo per passo tramite chiamate ricorsive nell'API di CVL, esplorando in modo consequenziale proprietà e relazioni. Per ogni terminal della combinazione, il sistema percorre uno per uno i salti della sua chain (sezione 5.1.3) e, a partire dall'entità radice, a ogni salto chiede all'API di proseguire dal nodo raggiunto fino a quel momento verso il nodo successivo, accumulando progressivamente lo stato della query in costruzione finché non raggiunge il nodo del terminal stesso. Questa procedura ripercorre esattamente il percorso che la fase di scoperta della mappa aveva già seguito per costruire la mappa iniziale (sezione 5.1.2); la differenza è che lì veniva fatto una volta per ogni proprietà isolata, mentre qui viene ripetuto per intero a ogni singolo record. Durante questa fase di costruzione online sono inoltre presenti alcuni meccanismi di resilienza, introdotti per gestire le condizioni anomale e gli eventuali problemi che possono verificarsi durante le chiamate all'API. Tali meccanismi si sono rivelati necessari nel corso dello sviluppo e vengono descritti di seguito.

- se l'API impiega troppo a rispondere, il fallimento è considerato transitorio, perché un tentativo successivo potrebbe ancora avere successo, e non viene memorizzato in nessuna cache;
- se invece risponde ma senza risultati utili, il terminal è considerato irraggiungibile in modo permanente, con due conseguenze distinte:
 - viene memorizzato in una cache condivisa così che i tentativi successivi lo scartino subito senza ripetere l'esplorazione;
 - il record in corso esclude dalla combinazione quel singolo terminal e la generazione prosegue con gli altri rimasti (esiste una modalità più rigida, mai usata in produzione, che interromperebbe l'intero tentativo al primo terminal irraggiungibile);
- indipendentemente dai primi due, per non ripetere l'intera esplorazione quando la stessa combinazione di terminal ricorre in record diversi, lo scheletro completo viene tenuto in una cache separata, indicizzata sulla combinazione esatta richiesta, e riusato direttamente se si ripresenta, con solo il riempimento dei valori concreti (sezione 5.1.7) che cambia da un record all'altro.

Il percorso offline

Con `offline_history` attivo, come nei run di produzione, tutta la camminata appena descritta viene saltata e lo scheletro non si costruisce esplorando dal vivo l'API di CVL, ma viene ricostruito interamente in locale a partire dai dati che la scoperta della mappa aveva già raccolto per ciascuna proprietà (sezione 5.1.2). Per ogni terminal coinvolto nella combinazione, il sistema ricrea la stessa sequenza di nodi intermedi che un'esplorazione dal vivo avrebbe prodotto, camminando sulla sequenza di salti già nota (registrata al momento della scoperta delle route, sezione 5.1.3) e usando i valori possibili già memorizzati per quel terminal, senza doverli richiedere di nuovo; con lo scheletro pronto, si arriva così direttamente all'unico passaggio che nessuno dei due percorsi può evitare, ovvero la chiamata finale descritta di seguito.

Listing 5.9: Lo scheletro appena ricostruito per i terminal `gender` e `birth date`, ancora privo di valori.

```
1 history = [  
2     {  
3         "query_nodes": [  
4             {  
5                 "predicate": {"label": "gender"},  
6                 "values": None,  
7                 "value_range": None  
8             },  
9             {  
10                "predicate": {"label": "birth date"},  
11                "values": None,  
12                "value_range": None  
13            }  
14        ]  
15    }  
16 ]
```

Indipendentemente dal percorso scelto per lo scheletro, il testo effettivo della query SPARQL viene sempre ottenuto con un'unica chiamata finale all'API, che traduce la struttura accumulata in una query eseguibile. Questo passaggio non è mai eseguito localmente, in nessuna delle due modalità, rappresentando così l'unica chiamata API fatta dal percorso offline. Su questa stessa chiamata finale interviene anche il quinto flag di listato 5.8, `validate_sparql`, ripreso e motivato più avanti insieme agli altri controlli di qualità applicati al risultato finale (sezione 5.1.9).

5.1.7 Riempimento dei valori

Una volta pronto lo scheletro, ogni suo nodo vuoto viene riempito con valori concreti scelti a caso dal pool associato al terminal corrispondente.

Listing 5.10: Riempimento di un nodo dello scheletro con valori concreti, distinguendo tra intervalli e insiemi discreti.

```
1 def fill_node(node, term, pool, min_values, max_values):  
2     k = random.randint(min_values, min(max_values, len(pool)))  
3     chosen = random.sample(pool, k)  
4  
5     if _is_range_datatype(term.get("datatype")):  
6         ordered = sorted(chosen, key=lambda v: v["value"])  
7         node["value_range"] = {  
8             "min": ordered[0]["value"],
```

```
9         "max": ordered[-1]["value"]
10     }
11     else:
12         node["values"] = chosen
```

Il numero di valori da usare per un dato vincolo è scelto a caso entro i limiti configurati (listato 5.10), evitando però di selezionare l'intero dominio possibile di un campo, perché un vincolo che copre tutti i valori ammessi non escluderebbe di fatto nessun paziente, risultando così privo di potere selettivo. Se il campo è una data o un valore numerico, i valori scelti vengono usati per costruire un intervallo anziché un elenco discreto.

Listing 5.11: Lo stesso scheletro di listato 5.9, dopo il riempimento: elenco discreto per il campo categoriale, intervallo per quello di tipo data.

```
1 history = [
2     {
3         "query_nodes": [
4             {
5                 "predicate": {"label": "gender"},
6                 "values": [{"value": "F", "label": "Female"}],
7                 "value_range": None
8             },
9             {
10                "predicate": {"label": "birth date"},
11                "values": None,
12                "value_range": {
13                    "min": "2016-01-01",
14                    "max": "2019-01-01"
15                }
16            }
17        ]
18    }
19 ]
```

5.1.8 Generazione del prompt e tracciamento degli span

Definiamo un *segmento* come un pezzo di testo accompagnato da un booleano che indica se il testo contenga un valore letterale, come una data o un codice, oppure solo una parola di struttura della frase come *with* o *and*. Con lo scheletro completo di valori, il prompt viene costruito ri assemblando i suoi nodi in una lista

di segmenti, concatenati uno via l'altro formando il prompt finale. La formulazione di ciascun segmento dipende da cosa contiene il nodo di origine:

- se il nodo ha valori scelti (sezione 5.1.7), il segmento inizia con *with* seguito dai valori stessi, uniti da *or* se sono due, o da una serie di virgole con un *or* finale se sono di più;
- se rappresenta un intervallo, diventa *between X and Y*;
- un nodo senza valori propri, che serve solo a passare al nodo successivo della chain senza introdurre un vincolo, produce invece un segmento con *which* al posto di *with*; un nodo *diagnosis* senza valori seguito da un nodo *stage* con valore *II* produce ad esempio "*which diagnosis with stage II*".

I segmenti di uno stesso passo della chain vengono uniti con un semplice *and*, e lo stesso connettivo lega tra loro i passi successivi, a partire da un'apertura fissa ("*Find all <entità>*").

Listing 5.12: Costruzione del prompt segmento per segmento, con tracciamento diretto della posizione di ciascun valore letterale.

```
1 def build_prompt_with_spans(root_label, history, skip_navigation=False):
2     segments = _prompt_segments(root_label, history, skip_navigation)
3     pieces, spans = [], []
4
5     for is_value, text in segments:
6         if is_value:
7             base = sum(len(p) for p in pieces)
8             spans.append((base, base + len(text)))
9             pieces.append(text)
10
11     return "".join(pieces), spans
```

Mentre i segmenti vengono concatenati uno alla volta, la posizione di ciascun valore inserito nel testo viene registrata. Determinare gli intervalli solo a partire dal prompt finale sarebbe infatti ambiguo nel caso in cui lo stesso valore comparisse, per coincidenza, anche in un'altra posizione della frase. Registrare le coordinate durante la costruzione del testo elimina quindi questa ambiguità senza richiedere controlli aggiuntivi. La funzione restituisce sempre la coppia (**prompt**, **value_spans**), ma solo più avanti, al momento di salvare il record completo, il

flag `include_value_spans` (sezione 5.1.6) decide se il campo `value_spans` venga scritto nell'output finale o scartato (sempre attivo nei run di produzione).

Applicata allo scheletro pieno di listato 5.11, la procedura produce il prompt *"Find all patient with gender Female and birth date between 2016-01-01 and 2019-01-01"*, con `value_spans` pari a `[(29, 35), (59, 69), (74, 84)]`, che corrispondono rispettivamente a *"Female"*, *"2016-01-01"* e *"2019-01-01"*.

5.1.9 Controlli di qualità e deduplicazione

Prima della persistenza su disco, ogni coppia prompt-query generata viene sottoposta a una sequenza di tre controlli di qualità indipendenti:

1. **Verifica dell'effettiva applicazione dei filtri:** Il primo controllo accerta che la query SPARQL generata dall'API contenga le clausole `FILTER` o `VALUES`, oppure almeno uno dei valori letterali campionati. Questo passaggio intercetta le risposte anomale del backend che, pur restituendo una sintassi valida, ignorano i vincoli richiesti. Una query priva di filtri risulterebbe semanticamente decontestualizzata rispetto al prompt associato, introducendo un esempio errato nel dataset. Qualora nessuna condizione sia soddisfatta, il record viene scartato con la motivazione esplicita *"the backend ignored our filters"*.

Listing 5.13: Verifica della presenza dei vincoli nella query SPARQL.

```
1 def _filters_landed(sparql, chosen_values):
2     if "FILTER" in sparql or "VALUES" in sparql:
3         return True
4     return any(v in sparql for v in chosen_values)
```

2. **Parsing sintattico deterministico:** Il secondo controllo valida la correttezza formale della query tramite il parser della libreria `rdflib`. L'obiettivo è prevenire l'inclusione di query malformate che insegnerebbero al modello linguistico una grammatica errata. Il controllo è subordinato al parametro di configurazione `validate_sparql` (listato 5.8), abilitato nei run di produzione. Gli esempi che falliscono l'analisi sintattica vengono scartati con il motivo *"the SPARQL does not parse (rdflib)"*.

Listing 5.14: Validazione sintattica condizionata della query SPARQL.

```
1 syntax_error = (  
2     validate.sparql_error(sparql)  
3     if self.config.get("validate_sparql", True)  
4     else None  
5 )
```

3. **Deduplicazione e rigenerazione dei valori:** L'ultimo controllo garantisce l'unicità dei dati mantenendo due tabelle hash in memoria: una contenente i prompt già accettati e una per gli digest calcolati sulle coppie prompt-query. In questo modo si evita che l'estrazione casuale generi combinazioni identiche di route e valori, le quali ridurrebbero la varietà reale del corpus. Qualora venga rilevato un prompt duplicato, il sistema esegue fino a 4 tentativi di rigenerazione dei valori (`VALUE_REROLLS`) prima di scartare definitivamente il record.

Le coppie che superano l'intero processo di validazione vengono accumulate in un buffer di memoria e scritte su disco a blocchi di 50 elementi (`BUFFER_SIZE`).

5.1.10 Generazione manuale a entità singola

L'architettura del sistema disaccoppia nettamente la logica del motore di generazione finora descritto dalla sua modalità di invocazione. La classe `BulkDatasetGenerator` è progettata per essere del tutto "inconsapevole" del contesto di esecuzione, essa riceve infatti una semplice configurazione ed esegue il flusso di generazione indipendentemente dall'interfaccia che la richiama.

Per consentire l'interazione diretta con questo motore, nell'ambito di questo lavoro di tesi è stato realizzato un pannello di controllo interattivo integrato nella piattaforma CVL tramite il framework Streamlit. Tale modulo fornisce un'interfaccia grafica per istanziare ed eseguire il generatore su una singola entità radice, mettendo a disposizione 5 sezioni principali:

- **Graph Map:** preconditione per tutto il resto, ha un pulsante che permette di caricare la mappa del grafo (sezione 5.1.2) e ne mostra un riepilogo (numero di entità, proprietà e relazioni scoperte) una volta completata.

- **Mode:** sceglie tra questa modalità a entità singola e la generazione automatica da file di tier (sezione 5.1.11).
- **Starting Entity:** un menu a tendina per scegliere l'unica entità radice da cui far partire la generazione.
- **Configuration:** quattro slider, uno per relazioni da attraversare, proprietà per percorso, percorsi simultanei e valori per proprietà.
- **Generation Mode e History building:** espongono direttamente, tramite interruttori e caselle di spunta, parametri già discussi altrove in questo capitolo, come la scelta tra campionamento e forza bruta e il numero di worker (sezioni 5.1.4 e 5.1.5), o i flag `offline_history`, `include_value_spans`, `skip_navigation_in_prompt` (sezione 5.1.6).

Gli estremi dei quattro slider di configurazione non sono fissi, ma vengono calcolati a runtime scoprendo, tramite `discover_routes_from_map` (sezione 5.1.3), quante route siano effettivamente raggiungibili da quell'entità con i limiti scelti.

Questa modalità non è pensata per costruire un dataset completo, ma per esplorare e calibrare. Al termine della generazione, infatti, il pannello non si limita a riportare un conteggio finale, ma mostra due schede separate:

- una con i record effettivamente creati, in un'anteprima tabellare di prompt e query
- una con quelli scartati, raggruppati per motivo di scarto esatto (tra cui i tre già discussi in sezione 5.1.9) con la possibilità di filtrare e ispezionare i singoli tentativi falliti per una singola categoria

È proprio questo riscontro dettagliato a rendere la modalità manuale utile a decidere quali parametri applicare a un tier con decine di entità e tempi di generazione elevati, dove un errore nei parametri rischierebbe di sprecare risorse di calcolo e prolungare inutilmente i tempi di attesa.

5.1.11 Configurazione dei tier da file esterno

La modalità interattiva descritta nella sezione precedente consente l'esplorazione di una singola entità alla volta; tuttavia, le diverse classi dello schema richiedono parametri di generazione differenziati in base alla loro rilevanza clinica, alla densità delle loro connessioni nel grafo e al volume di varianti da rappresentare.

Per gestire questa eterogeneità, il generatore supporta la definizione di file di configurazione esterni in cui le entità vengono raggruppate in insiemi omogenei di parametri, denominati *tier*. Tale astrazione evita l'applicazione di vincoli uniformi all'intero schema o di ripetitive generazioni manuali e permette di rendere le configurazioni riutilizzabili, tracciabili e facilmente modificabili. Per ogni entità appartenente a un tier, il motore `BulkDatasetGenerator` viene invocato applicando le impostazioni specifiche di quel gruppo e un limite massimo di campioni per entità (`sample_size_per_entity`). I record generati dalle singole entità vengono infine aggregati in un unico dataset primario per il successivo processo di suddivisione (sezione 5.1.12).

Nell'interfaccia Streamlit, la gestione dei tier è affidata a un modulo dedicato che consente il caricamento del file di configurazione, la verifica preventiva dei gruppi e il monitoraggio dell'avanzamento basato sul numero di entità elaborate. Al termine del processo, il sistema rende disponibile il dataset direttamente per il download, eliminando la necessità di accedere al filesystem del server ospitante.

Infine, per garantire la robustezza dell'esecuzione, la configurazione dei tier viene sottoposta a una procedura di validazione preventiva. Questa verifica accerta la presenza delle chiavi obbligatorie, la validità dei range parametrici e l'unicità dei nomi dei gruppi; un file non conforme viene rigettato all'atto del caricamento con un'indicazione puntuale dell'errore, prevenendo interruzioni anomale nelle fasi avanzate della generazione.

Listing 5.15: Esempio minimo di configurazione di un tier da file JSON esterno.

```
1  [
2    {
3      "name": "diagnosis_related",
4      "entities": ["Diagnosis", "TNMClassification"],
5      "sample_size_per_entity": 50,
6      "min_relations": 1,
7      "max_relations": 3,
```

```
8         "min_paths": 1,
9         "max_paths": 2,
10        "min_properties": 1,
11        "max_properties": 3,
12        "min_values": 1,
13        "max_values": 2
14    }
15]
```

5.1.12 Split del dataset

Una volta che tutte le entità dei tier configurati sono state processate e unite in un unico file, l'ultimo passo divide i record in train, validation e test.

Listing 5.16: Suddivisione del dataset in train, validation e test, stratificata per entità di origine.

```
1 for entity, recs in by_entity.items():
2     rng.shuffle(recs)
3     n = len(recs)
4
5     n_val = max(1, round(n * SPLIT_RATIOS["val"])) if n >= 10 else 0
6     n_test = max(1, round(n * SPLIT_RATIOS["test"])) if n >= 10 else 0
7     n_train = n - n_val - n_test
8
9     splits["train"].extend(recs[:n_train])
10    splits["val"].extend(recs[n_train:n_train + n_val])
11    splits["test"].extend(recs[n_train + n_val:])
```

I record vengono raggruppati per entità di origine e suddivisi separatamente per ciascun gruppo, con proporzioni 80/10/10 e seme fisso a 42 per la riproducibilità (listato 5.16), garantendo così che ogni entità sia rappresentata nella stessa proporzione in tutti e tre gli split, invece di rischiare che un'entità poco frequente finisca per puro caso quasi interamente in uno solo di essi. Le entità con meno di 10 record totali non vengono suddivise e finiscono interamente in train, perché un campione così piccolo non permetterebbe una validazione o un test statisticamente significativi.

Lo split avviene prima dello stadio di augmentation, non dopo, per una ragione precisa. Se l'augmentation venisse eseguita sull'intero dataset ancora unito, più varianti parafrasate dello stesso record sorgente potrebbero finire sparse su split diversi, contaminando la separazione tra train e test con esempi troppo simili tra

loro. Il generatore lo ricorda esplicitamente in un messaggio di log al termine dello split, indicando di eseguire la pipeline di augmentation separatamente su ciascuno dei tre file prodotti.

5.2 Implementazione dell'augmentation

La fase di *data augmentation* ha lo scopo di estendere la varietà linguistica dei prompt generati dal primo stadio della pipeline, garantendo al contempo che il significato semantico e i riferimenti puntuali alla query SPARQL sottostante rimangano inalterati (O2, RNF3).

Nelle prime fasi di esplorazione del progetto, la variazione stilistica è stata affidata direttamente a un modello linguistico, fornendo il testo in chiaro e richiedendone la rielaborazione in diversi registri linguistici (dallo stile clinico-formale a quello colloquiale). Per questo compito di generazione è stato impiegato il modello **Gemma4:e2B**², eseguito in locale tramite il runtime di inferenza Ollama³, selezionato in quanto rappresentativo di un ottimo compromesso tra qualità della riformulazione testuale, ridotto ingombro computazionale e celerità di inferenza nell'esecuzione locale.

Tuttavia, l'affidamento diretto al modello senza vincoli strutturali si è scontrato con i limiti pratici derivanti dall'impiego di un'architettura locale di taglia contenuta (V3), vincolo imposto dalla necessità di garantire il trattamento riservato dei dati sanitari su infrastruttura locale (V2):

1. **Alterazione e perdita di valori clinici (*hallucination*):** Sebbene capace di buona fluidità sintattica, il modello locale tendeva a modificare o omettere identificativi, date, codici clinici e valori booleani, compromettendo la corrispondenza univoca tra il prompt e i filtri della query SPARQL.
2. **Instabilità al crescere della complessità:** All'aumentare del numero di vincoli e valori presenti nella domanda originale, la capacità del modello di conservare l'intero contesto degradava, portando all'omissione involontaria di intere clausole.

²[\[https://ai.google.dev/gemma\]](https://ai.google.dev/gemma) (<https://ai.google.dev/gemma>)

³<https://ollama.com>

3. **Convergenza stilistica:** Lasciato libero di rielaborare il testo senza una guida rigida, il modello tendeva a riproporre le medesime strutture sintattiche e aperture di frase ricorrenti, vanificando l'obiettivo di varietà stilistica.

Per superare queste criticità e garantire un tasso di accettazione dei record economicamente sostenibile (RNF1), la pipeline di augmentation è stata completamente riprogettata attorno a un meccanismo rigoroso di **mascheramento dei valori letterali** (mediante placeholder opachi) associato a un ciclo iterativo di **generazione guidata, validazione e fallback**.

Nelle sottosezioni seguenti vengono descritte le componenti software che realizzano tale architettura.

5.2.1 Mascheramento

5.2.2 Mascheramento deterministico dei valori

Per isolare la struttura sintattica del prompt dai dati puntuali, la prima fase della pipeline di *augmentation* sostituisce ogni valore letterale con un identificativo opaco del tipo [VAL_XXXX], dove [XXXX], ovvero la componente alfanumerica, viene generata dinamicamente (listato 5.17).

L'alfabeto impiegato per la creazione dei token (TOKEN_ALPHABET) esclude deliberatamente i caratteri ambigui (I, L, 0, O, 1). Questa scelta nasce da un'evidenza emersa nelle prime fasi di test, i modelli di lingua di piccole dimensioni mostravano un'elevata frequenza di allucinazione e sostituzione indebita su certi caratteri, probabilmente a causa della loro somiglianza visiva nei vettori di embedding. I placeholder vengono generati all'interno di un insieme (**set**) per garantire l'assoluta unicità dei token all'interno dello stesso record.

La funzione **mask** applica le sostituzioni basandosi direttamente sugli intervalli posizionali (*spans*) calcolati dal generatore (sezione 5.1.8), evitando costose ed errate ricerche per sottostringa a posteriori. Infine, per la gestione dei placeholder vengono impiegate due espressioni regolari con scopi distinti:

- **Versione rigorosa (VAL_RE):** Riconosce esclusivamente la struttura esatta dei token generati, utilizzata per estrarre e ricomporre i valori autentici;

- **Versione permissiva (LOOSE_VAL_RE):** Rileva varianti sintatticamente storpiate o generate arbitrariamente, impiegata nella successiva fase di validazione per intercettare ed eliminare le allucinazioni del modello (sezione 5.2.7).

Listing 5.17: Generazione dei placeholder escludendo i caratteri ad alta frequenza di allucinazione e mascheramento basato su posizioni.

```
1 TOKEN_ALPHABET = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789"
2
3 def generate_placeholder_tokens(n):
4     tokens = set()
5     while len(tokens) < n:
6         token = "[VAL_" + "".join(random.choices(TOKEN_ALPHABET, k=4)) + "]"
7         tokens.add(token)
8     return list(tokens)
9
10 def mask(text, spans):
11     spans = sorted(spans)
12     tokens = generate_placeholder_tokens(len(spans))
13     mapping, pieces, cursor = {}, [], 0
14
15     for (start, end), placeholder in zip(spans, tokens):
16         pieces.append(text[cursor:start])
17         mapping[placeholder] = text[start:end]
18         pieces.append(placeholder)
19         cursor = end
20
21     pieces.append(text[cursor:])
22     return "".join(pieces), mapping
```

5.2.3 Semplificazione booleana e generazione degli hint

Come anticipato nella fase di design, non tutti i placeholder mascherati richiedono l'elaborazione da parte del modello linguistico. I valori booleani vengono intercettati ed eliminati prima della fase di inferenza tramite la funzione `simplify_boolean_phrasing` (listato 5.18).

Listing 5.18: Trascrizione proposizionale ed eliminazione preventiva dei placeholder booleani.

```
1 def simplify_boolean_phrasing(text, mapping):
2     new_mapping = dict(mapping)
3     for placeholder, value in mapping.items():
4         if value.lower() not in {"true", "false"}:
```

```
5         continue
6
7         is_true = value.lower() == "true"
8         repl = r"with \1" if is_true else r"without \1"
9         esc = re.escape(placeholder)
10        stop = r"(?:with|which|and)"
11
12        which_re = re.compile(
13            rf"\bwhich ((?:?!{stop}\b)\w+ ?){1,5}} is {esc}",
14            re.IGNORECASE
15        )
16        with_re = re.compile(
17            rf"\bwith ((?:?!{stop}\b)\w+ ?){1,5}} {esc}",
18            re.IGNORECASE
19        )
20
21        if which_re.search(text):
22            text = which_re.sub(repl, text)
23            del new_mapping[placeholder]
24        elif with_re.search(text):
25            text = with_re.sub(repl, text)
26            del new_mapping[placeholder]
27
28        return text, new_mapping
```

La procedura analizza la struttura sintattica che circonda il token: un valore booleano affermativo (**true**) viene convertito nella preposizione *"with"* seguita dal termine clinico di riferimento, mentre un valore negativo (**false**) viene convertito in *"without"*. L'operazione rimuove il token sia dal testo sia dal dizionario di mappatura (**new_mapping**), sollevando le fasi successive della pipeline dalla necessità di tracciare o ripristinare il valore.

Per tutti i restanti valori mascherati non booleani, il modulo invoca la funzione **build_hint_map**, la quale costruisce un suggerimento in linguaggio naturale (*hint*) seguendo una gerarchia di regole deterministiche ad alta priorità:

- **Date in formato ISO:** Espresse in forma testuale estesa per preservare la naturalezza della frase;
- **Valori numerici interi (< 1000):** Convertiti dalla notazione cifrata alla corrispondente forma letterale;
- **Etichette composte da trattino:** Semplificate isolando la componente terminologica specifica;

- **Fallback su modello linguistico:** Qualora nessuna delle trasformazioni euristiche sopra descritte sia applicabile, viene eseguita un'interrogazione mirata al modello per estrarre un sinonimo di uso comune nel dominio.

Il suggerimento così generato viene applicato, con una probabilità configurabile, sia durante la fase di de-mascheramento e validazione finale (sezione 5.2.7), sia come guida contestuale nella modalità di generazione priva di mascheramento (sezione 5.2.10).

5.2.4 Suddivisione in chunk per record complessi

All'aumentare della densità di placeholder in un singolo prompt, le prestazioni del modello linguistico degradano visibilmente, portando a omissioni o alla mancata rielaborazione di intere clausole. Per mitigare questo fenomeno, i record con un numero elevato di valori mascherati vengono suddivisi in più porzioni indipendenti (*chunk*) prima di essere inoltrati al modulo di generazione (listato 5.19).

Listing 5.19: Suddivisione in chunk del testo mascherato

```
1 MAX_VALUES_PER_CHUNK = 12
2
3 def split_into_chunks(masked_text, mapping, max_values_per_chunk):
4     clauses = _protect_between(masked_text).split(CLAUSE_SEP)
5     chunks = [[]]
6     count_in_chunk = 0
7
8     for clause in clauses:
9         n_values = len(VALL_RE.findall(clause))
10        if count_in_chunk > 0 and count_in_chunk + n_values > max_values_per_chunk:
11            :
12            chunks.append([])
13            count_in_chunk = 0
14            chunks[-1].append(clause)
15            count_in_chunk += n_values
16
17    return [_restore_between(CLAUSE_SEP.join(c)) for c in chunks if c]
```

L'algoritmo segmenta il testo basandosi sulle congiunzioni coordinanti ("and"), aggregando le clausole in un unico chunk fino al raggiungimento della soglia massima di 12 token (MAX_VALUES_PER_CHUNK). Per evitare la frammentazione incoerente di costrutti sintattici indivisibili, le funzioni `_protect_between` e `_restore_between`

mascherano temporaneamente i connettivi interni alle espressioni di intervallo (*"between X and Y"*) sostituendoli con un carattere di controllo esadecimale (NUL, ASCII 0x00).

I record il cui conteggio di valori rientra nella soglia limite non subiscono alcuna suddivisione. A titolo esemplificativo, nella produzione del dataset da 1 331 record generato con vincolo a 4 proprietà (sezione 5.1.11):

- l'**80,6%** dei casi (1 073 record) ha prodotto un singolo chunk;
- il restante **19,4%** (258 record) è stato ripartito esattamente su due chunk;
- nessun record ha necessitato di una frammentazione superiore a due chunk, con un caso limite di 20 token totali suddivisi in due sotto-porzioni rispettivamente da 11 e 9 valori.

5.2.5 Ingegneria dei prompt e direttive di riformulazione

La qualità e la robustezza delle parafrasi prodotte da **Gemma4:e2B** dipendono in modo cruciale dalla struttura dei template di prompt inoltrati al modello. Per guidare la generazione garantendo al contempo l'invarianza semantica e sintattica, sono stati progettati tre template principali (listato 5.20):

- **PROMPT_TEMPLATE:** Utilizzato per la rielaborazione completa dei prompt mascherati o del primo chunk. Definisce le regole tassative per la conservazione letterale dei token [VAL.XXXX], impone lo stile obiettivo, forza l'eliminazione delle ridondanze tipiche dei template rigidi (es. la semplificazione delle catene di navigazione relazionale del grafo) e vieta prefissi o metatesto nell'output.
- **FRAGMENT_TEMPLATE:** Impiegato per i chunk successivi al primo (**as_fragment=True**). Omette le direttive stilistiche globali per mantenere il registro già stabilito, vieta la punteggiatura finale e impedisce l'inserimento della congiunzione coordinante iniziale (*"and"*), affidandone la gestione al modulo di ricomposizione.
- **NO_MASK_TEMPLATE:** Utilizzato nella modalità di generazione priva di mascheramento dei valori (sezione 5.2.10), in cui i valori letterali vengono forniti in chiaro affiancati da suggerimenti stilistici facoltativi (*value hints*).

Listing 5.20: Struttura dei template di prompt e funzioni per l'iniezione dinamica del contesto di feedback e storico.

```

1 PROMPT_TEMPLATE = """Rewrite the following query request as a single reformulation
  in {language}.
2
3 Rules:
4 - The text below contains placeholders in square brackets, each one unique: copy
  each one EXACTLY as it appears, character-for-character.
5 - Every single placeholder in the original MUST appear, unchanged, in the
  reformulation.
6 - The original has a rigid template style: REMOVE repetition and phrase naturally.
7 - Compress awkward navigation chains (e.g. "which has height with metric unit [VAL
  ]") into natural outcomes ("with a height measured in [VAL]") when
  intermediate steps add no value.
8 - Write it as {style}.
9 - Actively vary fact ordering: do not always follow the original traversal order.
10 {avoid_history_section}- Keep all clinical meaning exactly intact.
11 - Output ONLY the reformulation, nothing else.
12 {avoid_section}
13 Original: {question}
14
15 Reformulation: """
16
17 def build_avoid_history_section(previous_texts: list) -> str:
18     if not previous_texts:
19         return ""
20     items = "\n".join(f' - "{t}"' for t in previous_texts)
21     return (
22         "- Do NOT reuse the wording, structure, or vocabulary of these previous "
23         f"reformulations written in this same style:\n{items}\n"
24     )
25
26 def build_avoid_section(avoid_text: str, reason: str = None) -> str:
27     if not avoid_text:
28         return ""
29     reason_line = f" Specifically, the problem was: {reason}." if reason else ""
30     return (
31         f"\nA previous attempt was rejected.{reason_line} Do NOT repeat this
32         mistake, "
33         f"and do NOT repeat this wording - write something meaningfully different
34         :\n\"{avoid_text}\""
35     )

```

Come mostrato nelle funzioni di supporto `build_avoid_history_section` e `build_avoid_section`, il sistema arricchisce dinamicamente il prompt di input:

1. **Preventivo anti-convergenza:** Iniettando le strutture sintattiche recentemente generate per lo stesso stile (`avoid_history`), costringendo il modello

a esplorare percorsi sintattici alternativi.

2. **Feedback reattivo negli errori:** Inserendo nel ciclo di retry il testo scartato e la motivazione esplicita fornita dal validatore (`avoid_reason`), trasformando la validazione in un meccanismo di apprendimento in-context guidato.

5.2.6 Generazione iterativa: stile, retry e fallback

Sfruttando le strutture di prompt e le funzioni di iniezione del contesto descritte nella sezione precedente (sezione 5.2.5), ogni sotto-porzione (*chunk*) e ogni successiva variante completa vengono elaborate mediante un ciclo di generazione adattivo. La funzione `generate_piece` (listato 5.21) orchestra i tentativi di inferenza combinando un campionario a temperatura variabile, l'iniezione dinamica del feedback e un meccanismo di fallback di sicurezza.

Listing 5.21: Ciclo di generazione e validazione iterativa per un singolo chunk con gestione del fallback.

```
1 def generate_piece(  
2     model, piece_text, piece_mapping, style_desc, max_attempts,  
3     base_temp, max_temp, avoid_history, indent, label,  
4     avoid_text=None, avoid_reason=None, as_fragment=False,  
5     language="English"  
6 ):  
7     total_elapsed = 0.0  
8     for attempt in range(max_attempts):  
9         temperature = temperature_for_attempt(  
10             attempt, max_attempts, base_temp, max_temp  
11         )  
12  
13         candidate, elapsed = (  
14             paraphrase_fragment(  
15                 model, piece_text, piece_mapping, temperature,  
16                 language, avoid_text, avoid_reason  
17             )  
18             if as_fragment else  
19             paraphrase_one(  
20                 model, piece_text, piece_mapping, style_desc,  
21                 temperature, language, avoid_text, avoid_reason,  
22                 avoid_history if attempt == 0 else None  
23             )  
24         )  
25
```

```
26     total_elapsed += elapsed
27     reason = validation_reason(candidate, piece_mapping, [])
28
29     if reason is None:
30         return candidate, total_elapsed, False
31
32     avoid_text, avoid_reason = candidate, reason
33
34     return piece_text, total_elapsed, True
```

L'esecuzione del ciclo di generazione si articola sulle seguenti logiche operative:

- **Campionamento adattivo e ciclo di feedback:** Per ciascun tentativo (fino a un massimo di `max_attempts`, pari a 5 di default), la funzione `temperature_for_attempt` calcola una temperatura di inferenza cresciuta linearmente da `base_temp` a `max_temp`. Se il candidato prodotto viene respinto dal validatore (`validation_reason`), le variabili `avoid_text` e `avoid_reason` registrano l'output errato e il relativo motivo; tali informazioni vengono automaticamente convertite da `build_avoid_section` ed iniettate nel prompt del tentativo successivo, costringendo il modello a esplorare lo spazio latente in direzioni alternative.
- **Differenziazione tra primo chunk e frammenti successivi:** La generazione applica una diversa strategia a seconda della posizione del chunk. Il primo pezzo viene inviato tramite `paraphrase_one` utilizzando il `PROMPT_TEMPLATE` completo, comprensivo della descrizione dello stile e della cronologia delle strutture da evitare (`avoid_history`). I chunk successivi vengono invece elaborati tramite `paraphrase_fragment` (`as_fragment=True`), inoltrando il `FRAGMENT_TEMPLATE`; questa versione ridotta omette le direttive stilistiche globali e impedisce l'inserimento di congiunzioni o punteggiatura finale, garantendo la continuità sintattica con il primo frammento.
- **Garantismo e fallback deterministico:** Qualora tutti i tentativi consentiti falliscano a causa del persistere di violazioni sintattiche o di allucinazioni sui token, il sistema evita di forzare un testo degradato o incompleto. La funzione restituisce il testo mascherato originale non modificato (`piece_text`), impostando il flag di fallback su `True`. In questo modo, la pipeline garantisce la continuità dell'esecuzione senza introdurre errori nel dataset.

I controlli effettuati all'interno di `generate_piece` sono strettamente focalizzati sulla validità strutturale dei singoli chunk (come l'integrità dei token di mascheramento). I controlli globali, tra cui il calcolo della similarità sintattica rispetto ad altre varianti e il ripristino finale dei valori, vengono posticipati alla fase di ricomposizione e validazione dell'intero record (sezione 5.2.7).

5.2.7 Validazione: due funzioni per due modalità

La validazione di un candidato applica fino a tre criteri in sequenza all'interno di un'unica funzione (listato 5.22), più un quarto controllo esterno sulla ripetizione dell'apertura di frase.

Listing 5.22: La funzione di validazione condivisa da chunk e varianti complete, con soglia di somiglianza adattiva.

```
1 def validation_reason(candidate, mapping, accepted_so_far, max_similarity=None):
2     missing = [p for p in mapping if p not in candidate]
3     if missing:
4         return f"you dropped the placeholder(s) {'', '.join(missing)} - every
5             single one must appear unchanged"
6
7     malformed = sorted(set(m for m in LOOSE_VAL_RE.findall(candidate) if m not in
8         mapping))
9     if malformed:
10        return f"you wrote malformed or invented placeholder-like text: {'', '.join
11            (malformed)} - placeholders must be copied exactly, never altered"
12
13    threshold = max_similarity if max_similarity is not None else
14        similarity_threshold_for(candidate)
15
16    for prev in accepted_so_far:
17        ratio = difflib.SequenceMatcher(None, candidate.lower(), prev.lower()).
18            ratio()
19        if ratio >= threshold:
20            return f"this wording is too similar ({ratio:.0%} match) to a variant
21                already produced for this same request"
22
23    return None
```

Si parla di soglia di somiglianza adattiva poichè il controllo di somiglianza `difflib.SequenceMatcher`⁴, della libreria standard, varia a seconda della lunghezza del testo:

⁴<https://docs.python.org/3/library/difflib.html>

- più permissiva (fino al 97%) per frasi brevi
- più severa (85%) per frasi lunghe

Questa piccola differenziazione è stata valutata come necessaria durante l'implementazione, altrimenti testi corti sarebbero quasi sempre stati scartati come duplicati per mancanza di margine di variazione lessicale.

Per la modalità di generazione priva di mascheramento (sezione 5.2.10), il sistema adotta una funzione di validazione dedicata, caratterizzata da un criterio di verifica dei valori più flessibile. In questo scenario, poiché il modello linguistico opera direttamente sui dati in chiaro con la facoltà esplicita di applicarvi varianti stilistiche, un valore letterale non viene classificato come omesso qualora compaia nel testo sotto forma di suggerimento naturale (*value hint*) o come sinonimo plausibile, valutato mediante un confronto *case-insensitive*.

5.2.8 Rotazione degli stili e memoria anti-ripetizione

La diversificazione stilistica tra le differenti varianti di uno stesso record è gestita dalla classe `StyleTracker` (listato 5.23), la quale assegna i registri espressivi in sequenza ed esegue un tracciamento a finestra scorrevole delle generazioni recenti.

Listing 5.23: Assegnazione ciclica degli stili e tracciamento a finestra delle formulazioni e delle aperture recenti.

```
1 class StyleTracker:
2     def __init__(
3         self, styles=None, recent_size=3, max_chars=160, opener_window=12
4     ):
5         self._styles = styles if styles is not None else STYLES
6         self._style_cycle = itertools.cycle(self._styles)
7         self._recent_texts = {tag: [] for tag, _ in self._styles}
8         self._opener_history = {tag: [] for tag, _ in self._styles}
9         self._recent_size = recent_size
10        self._max_chars = max_chars
11        self._opener_window = opener_window
12
13    def next_style(self):
14        return next(self._style_cycle)
15
16    def remember(self, style_tag, text):
17        sanitized = VAL_RE.sub("...", text)
18        preview = (
```

```
19         sanitized if len(sanitized) <= self._max_chars
20         else sanitized[:self._max_chars - 3] + "... "
21     )
22
23     bucket = self._recent_texts.setdefault(style_tag, [])
24     bucket.append(preview)
25     del bucket[:-self._recent_size]
26
27     openers = self._opener_history.setdefault(style_tag, [])
28     openers.append(opener(text))
29     del openers[:-self._opener_window]
```

L'assegnazione dei dieci registri linguistici predefiniti (spazianti dal tono tecnico-database a quello colloquiale o statistico) avviene deterministicamente mediante iterazione circolare (`itertools.cycle`⁵). Tale scelta previene i sbilanciamenti propri di un campionamento casuale, assicurando una distribuzione uniforme dei registri sull'intero dataset.

Il metodo `remember` viene invocato esclusivamente a seguito dell'accettazione di una variante, aggiornando due strutture dati distinte per ciascun registro:

- **Storico delle formulazioni complete (`_recent_texts`):** Mantiene una finestra delle ultime 3 frasi generate. I token di mascheramento vengono preventivamente sostituiti con un punto di sospensione ("`...`") per impedire che gli identificativi opachi alterino la valutazione della similarità testuale.
- **Storico delle aperture sintattiche (`_opener_history`):** Conserva una finestra più estesa, pari alle ultime 12 sequenze di avvio (`openers`). Trattandosi di un'informazione a minore entropia rispetto alla frase intera, richiede uno storico più ampio per intercettare e neutralizzare pattern di incipit ricorrenti.

Il disaccoppiamento dei due tracciamenti in dizionari separati consente di applicare metriche di controllo e orizzonti temporali differenti, evitando riduzioni o troncamenti artificiali della memoria del modello.

5.2.9 Statistiche di qualità e resilienza

Al fine di monitorare la qualità del processo di sintesi e garantire un confronto tra le generazioni, ciascuna esecuzione della pipeline accumula un insieme di metriche

⁵<https://docs.python.org/3/library/itertools.html#itertools.cycle>

strutturate. Tali contatori permettono di tracciare la resa qualitativa delle varianti generate, distinguendo le formulazioni prodotte con successo da quelle derivanti da un meccanismo di fallback o dai fallimenti definitivi (listato 5.24).

Listing 5.24: Calcolo della resa complessiva del processo di generazione e serializzazione delle metriche di qualità.

```
1 def to_dict(self):
2     slots_failed = len(self.failed_slots)
3     yield_rate = (
4         (1 - (slots_failed / self.total_slots))
5         if self.total_slots else None
6     )
7     return {
8         "total_slots": self.total_slots,
9         "slots_failed": slots_failed,
10        "yield_rate": yield_rate,
11        "fallback_count": self.fallback_count,
12        "style_attempts": dict(self.style_attempts),
13        "style_accepted": dict(self.style_accepted),
14        "failed_slots": self.failed_slots,
15    }
```

Il monitoraggio delle prestazioni si basa su tre indicatori principali:

- **Tasso di resa (yield_rate):** Esprime la quota di varianti generate con successo rispetto al totale degli slot richiesti (`total_slots`).
- **Conteggio dei fallback (fallback_count):** Traccia le varianti valide in cui almeno un chunk non è stato rielaborato dal modello ed è stato ripristinato nella forma originale per garantire la correttezza del record.
- **Slot falliti (failed_slots):** Registra le richieste di variante per le quali, al termine di tutti i tentativi di rigenerazione previsti, non è stato possibile produrre un testo conforme. Per ciascun fallimento, la struttura dati conserva il tracciamento dettagliato dei tentativi e le relative cause di rifiuto.

Per soddisfare i requisiti di tolleranza ai guasti (RNF2), le statistiche vengono serializzate su disco a seguito dell'elaborazione di ogni singolo record, in questo modo, in caso di interruzione imprevista dell'esecuzione, un eventuale riavvio della pipeline non azzerà i contatori ma riprende lo stato accumulato. Parallelamente, un meccanismo di ripresa automatica analizza i dati già salvati, ricostruendo l'elenco

degli identificativi (`source_id`) completati ed evitando la rielaborazione ridondante dei record.

5.2.10 La modalità senza mascheramento

Come anticipato il sistema conserva, per ciascun record, anche un percorso di generazione alternativo che sottopone il testo intero privo di mascheramento al modello con i valori veri elencati esplicitamente, senza alcuna suddivisione in chunk (listato 5.25).

Listing 5.25: Implementazione della modalità baseline senza mascheramento per il confronto sperimentale.

```
1 def generate_variant_no_mask(  
2     model, text, values, style_tag, style_desc,  
3     max_attempts, base_temp, max_temp, avoid_history,  
4     slot_label, avoid_text=None, avoid_reason=None,  
5     language="English", hints=None  
6 ):  
7     # Deliberately no chunking here: this mode exists to reproduce  
8     # the pre-masking, single-shot behavior for direct comparison,  
9     # not to be a second production path.  
10    hints = hints or {}  
11    identity_mapping = {v: v for v in values}  
12    value_list = ", ".join(f'"{v}"' for v in values)  
13  
14    # ... ciclo di retry e formattazione del prompt ...  
15  
16    reason = validation_reason_with_hints(  
17        candidate, values, hints, []  
18    )
```

Durante la produzione del dataset questo tipo di generazione non è stata utilizzata, rappresenta quindi non un secondo percorso di produzione, ma un termine di paragone diretto per valutare il beneficio del mascheramento (capitolo 6). L'evidenza raccolta durante un'esecuzione diagnostica su questa modalità conferma empiricamente le scelte progettuali presentate nel capitolo 4. Su un totale di 41 rifiuti registrati nel log di un test condotto su due record, ben 36 casi (pari all'87,8%) sono stati causati dalla perdita o dall'alterazione dei valori letterali da parte del modello. Tale criticità viene resa strutturalmente impossibile dall'approccio ba-

sato su mascheramento, in quanto il modello non ha mai accesso ai valori reali durante la fase di riformulazione.

Capitolo 6

Valutazione

6.1 Testing

6.1.1 Problemi di qualità riscontrati e diagnosi delle cause

Nel corso dello sviluppo e delle fasi di testing sono stati individuati e analizzati cinque problemi di qualità principali (figura 6.1), di cui le relative proposte di risoluzione sono discusse nel capitolo 7.

1. **Scambio di acronimi clinici:** In un singolo caso, il modello di generazione dei sinonimi ha sostituito l'acronimo **ECOG** (relativo alla scala di stato funzionale oncologico) con la stringa riferita all'esame di ecocardiogramma. L'errore deriva dai limiti del modello locale di piccole dimensioni, che ha associato i due termini, basandosi con tutta probabilità, esclusivamente sulla somiglianza visiva del nome.
2. **Allucinazione dei nomi di patologia sui codici ICD-10:** In 47 record, un medesimo codice ICD-10 è stato tradotto in nomi di patologie errate ed incoerenti tra le diverse formulazioni. La causa era un bug nella regex del filtro di sicurezza dei sinonimi, il quale escludeva i valori nel formato **PREFIX:CODE** se il prefisso era alfabetico (come per lo standard SNOMED¹), ma non intercettava il prefisso ICD-10 perché contenente cifre. Il filtro

¹<https://www.snomed.org/>

di parsing è stato corretto, correggendo però solo in modo parziale il bug in questione.

3. **Formattazione rigida e non naturale:** Durante l'ispezione manuale dei set di validazione e test, si è riscontrata la presenza di frasi formulate come liste di chiavi-valore o vere e proprie query anziché testo fluido. Il problema non dipendeva dal modello linguistico, ma dalla struttura troppo rigida degli stili di frase scritti manualmente, che sono stati di conseguenza rielaborati.
4. **Mancanza di connettivo negli intervalli di date:** In 55 occorrenze, due valori di data adiacenti sono stati affiancati senza alcun separatore (es. "2020-01-01 2021-01-01"), generando stringhe ambigue. La causa era un'omissione nella procedura di assemblaggio del testo, che non prevedeva l'inserimento della congiunzione tra due campi di tipo data consecutivi; il generatore è stato aggiornato per inserirla automaticamente.
5. **Presenza del valore nullo "nan":** In 7 casi è stata rilevata la stringa letterale "nan" indicata come unità di misura. Il comportamento è una probabile conseguenza del mascheramento deterministico, che preserva integralmente le stringhe ricevute in input senza modificarle. Il difetto risiederebbe quindi nella presenza di dati mancanti non filtrati a monte nel dataset primario.

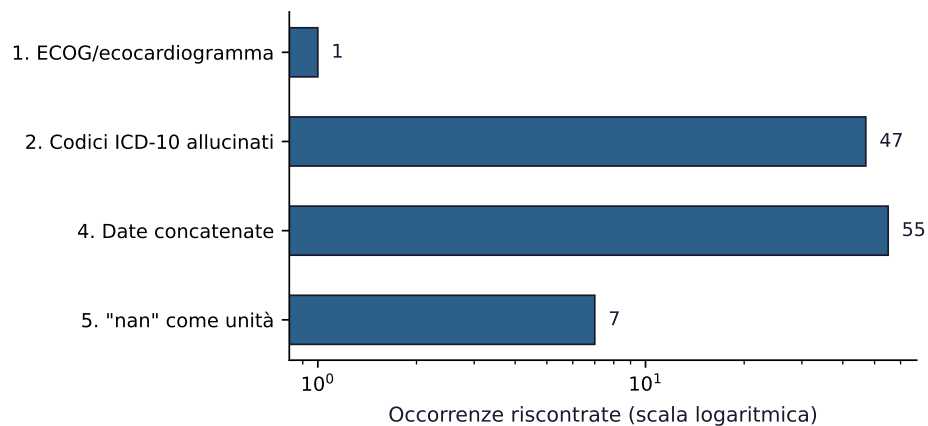


Figura 6.1: Distribuzione delle occorrenze per i quattro problemi quantificabili (il problema di sintassi non naturale, individuato tramite ispezione manuale, non compare in figura).

Comportamenti anomali valutati come ammissibili.

Due ulteriori fenomeni, inizialmente segnalati come bug, sono stati ri-classificati come variazioni accettabili:

- **Uso di preposizioni non canoniche (8 casi):** L'impiego di *"at"* al posto di *"on"* per le linee di terapia è stato considerato una naturale micro-varietà del linguaggio umano, utile ad aumentare la varianza stilistica.
- **Collisione di vincoli incompatibili (295 casi):** La generazione di query con filtri reciprocamente escludenti (es. due date di decesso discordanti nella stessa query) deriva dal campionamento indipendente di percorsi relazionali paralleli. Poiché rispecchiano interrogazioni reali ma errate che un utente sbadato può porre, i record non costituiscono un difetto.

6.1.2 Confronto tra generazione con e senza mascheramento

Per quantificare l'impatto del mascheramento sulla qualità delle variazioni stilistiche, è stato condotto un test comparativo rispetto alla modalità no-masking, la cui architettura è stata documentata nella sezione 5.2.10. L'esperimento ha posto a confronto le due strategie su un campione di 100 record sorgente, selezionati a complessità combinatoria crescente (da un singolo valore vincolato fino a 20 valori simultanei per query).

Metrica / Criterio di Valutazione	Con Mascheramento	Senza Mascheramento
Slot di generazione totali	330	330
Tasso di successo globale	98,2% (324/330)	89,1% (294/330)
Tasso di allucinazione sui valori	0,0% (0/330)	8,2% (27/330)
Causa primaria di rigetto	Similarità parziale (< 90%)	Duplicazione identica (100%)
Preservazione valori letterali	Garantita per costruzione	Modello-dipendente

Tabella 6.1: Confronto sperimentale tra la generazione con e senza mascheramento.

Come evidenziato nella tabella 6.1, la valutazione su 330 slot complessivi mostra un divario prestazionale netto a livello di tasso di successo, in cui la modalità mascherata raggiunge il **98,2%**, contro l'**89,1%** ottenuto dalla generazione senza mascheramento.

L'analisi dei log di scarto evidenzia due dinamiche qualitativamente differenti:

- **Modalità senza mascheramento:** Il 10,9% dei fallimenti totali è imputabile in massima parte (8,2%) a violazioni dell'integrità dei dati (alterazione o omissione dei valori letterali) e in misura minore (2,7%) a un collasso sintattico, in cui il modello ha rigenerato formulazioni identiche al 100% a quelle già accettate.
- **Modalità con mascheramento:** I soli 6 rigetti registrati (1,8%) sono derivati esclusivamente dal superamento della soglia di similarità adattiva su frasi brevi, senza alcuna corruzione dei token di mascheramento.

Questo comportamento è in gran parte riconducibile alle dimensioni contenute del modello linguistico impiegato (Gemma4:2B). Infatti architetture locali di piccola taglia, quando sollecitate a riformulare un testo mantenendo visibili i dati in chiaro, faticano a disaccoppiare la struttura sintattica dal contenuto informativo; di conseguenza, tendono a collassare su pattern espressivi rigidi o ad allucinare le

stringhe letterali. In questo modo abbiamo dimostrato che l'introduzione dei placeholder opachi solleva il modello dal carico di memoria sui dati, consentendogli di concentrare la capacità parametrica residuale unicamente sulla variazione stilistica della frase.

6.1.3 Statistiche del dataset ed analisi delle distribuzioni

Il corpus finale è stato suddiviso in tre insiemi disgiunti destinati ad addestramento, validazione e test secondo la procedura di partizione per entità descritta nella sezione 5.1.12. In seguito alla generazione vi è stata una verifica sugli identificativi univoci, attestando così l'assenza totale di sovrapposizioni tra i tre insiemi, garantendo la misurazione della reale capacità di generalizzazione del modello.

Split	Record Sorgente	Varianti Totali	Densità Media	Tasso Fallback
Train	3 443	9 976	2,90 var/rec	0,58% (58/9 976)
Validation	428	2 022	4,72 var/rec	0,35% (7/2 022)
Test	428	2 023	4,73 var/rec	0,35% (7/2 023)
Totale	4 299	14 021	–	0,51% (72/14 021)

Tabella 6.2: Riepilogo quantitativo del dataset suddiviso per split e relative metriche di densità e fallback.

Come evidenziato nella tabella 6.2, si osserva una marcata differenza di densità di parafrasi tra il set di addestramento e gli insiemi di valutazione. Nel set di addestramento, il numero di varianti per record è governato dalla formula adattiva proporzionale alla complessità dei filtri (media di 2,90), una scelta progettuale mirata a contenere la ridondanza complessiva senza saturare il tempo di calcolo. Al contrario, per validation e test è stato imposto un target fisso di 5 varianti per record (raggiunto con una media di 4,72 e 4,73), assicurando un banco di prova ricco e diversificato per la misurazione delle prestazioni.

In tutti e tre gli insiemi, l'incidenza delle varianti non riformulate rispetto all'originale (*fallback*) si mantiene estremamente contenuta ($< 0,6\%$), attestando la capacità del sistema di completare la generazione senza dover ricorrere alla frase di origine.

Analisi della distribuzione stilistica.

L'assegnazione degli stili linguistici avvenga in sequenza circolare deterministica (`itertools.cycle`); tuttavia, la distribuzione finale delle 9976 varianti dell'insieme di addestramento (figura 6.2) mostra uno sbilanciamento misurabile tra i dieci registri: da un massimo di 1237 occorrenze per *technical_sql* a un minimo di 569 per *generational* (rapporto di circa 2,2 a 1).

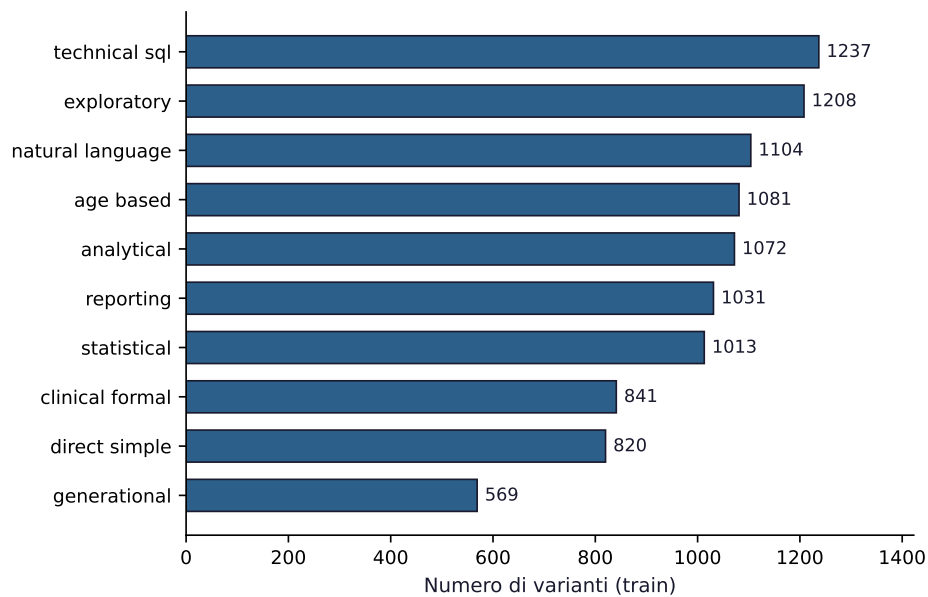


Figura 6.2: Distribuzione delle varianti accettate tra i dieci registri stilistici nell'insieme di addestramento.

Tale discrepanza nella distribuzione degli stili non deriva da un difetto di campionamento, ma dall'azione dei filtri di validazione della pipeline. Direttive stilistiche più vincolanti riducono lo spazio di variazione sintattica del modello, portandolo a convergere su strutture e lessico ricorrenti e, di conseguenza, i meccanismi di memoria anti-ripetizione (sezione 5.2.8) e il controllo di similarità adattiva (sezione 5.2.7) intercettano tali ridondanze scartando un numero maggiore di tentativi per garantire la varietà del dataset.

6.1.4 Metodologia

La valutazione del modello ha seguito un approccio strutturato in tre componenti principali:

1. **Scelta e addestramento del modello:** È stato selezionato **Gemma 3 270M**, un modello ultraleggero che garantisce l'esecuzione in locale (rispettando V2 e V3). Il modello è stato addestrato tramite la tecnica LoRA (sezione 2.5.3), aggiornando solamente lo 0,5% dei parametri totali per contenere i costi computazionali.
2. **Definizione della metrica:** Le prestazioni sono state misurate calcolando l'**Exact Match (EM)**. Questa metrica indica la percentuale di query SPARQL generate che risultano identiche, carattere per carattere, alla query target di riferimento.
3. **Allestimento dei dati di test:** La valutazione è stata condotta su due insiemi separati da 100 esempi ciascuno.
 - (a) *trainstyle-test (in-distribution)*, utilizza gli stessi registri stilistici visti in addestramento per verificare l'apprendimento delle regole base
 - (b) *claudetest (out-of-distribution)*, utilizza formulazioni generate da un modello esterno (*Claude 3.5 Sonnet*) per testare la capacità del sistema di comprendere frasi e costrutti linguistici del tutto nuovi

6.1.5 Selezione del checkpoint e prevenzione dell'overfitting

Nelle architetture di tipo Text-to-SPARQL, il numero di iterazioni di addestramento gioca un ruolo critico; un addestramento insufficiente non garantisce la memorizzazione dello schema, mentre un'esposizione prolungata conduce rapidamente al collasso sui pattern di input (*overfitting*).

Per identificare il regime ottimale, è stato condotto un addestramento esteso su 15 000 iterazioni con monitoraggio continuo della *validation loss* (figura 6.3).

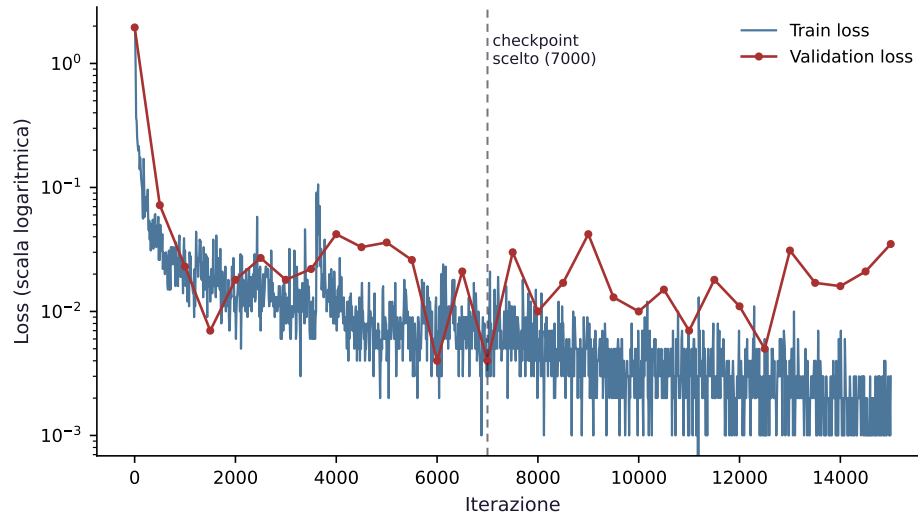


Figura 6.3: Andamento della train e validation loss durante le 15 000 iterazioni.

Invece di adottare il checkpoint finale (iterazione 15 000), la selezione metodologica è ricaduta rigorosamente sul punto di minimo della perdita di validazione, individuato all'iterazione 7 000 (usando dei checkpoint ogni 500 iterazioni). Questa scelta ha prodotto una svolta determinante nelle prestazioni, portando l'Exact Match sull'insieme in-distribution dal **15%** del checkpoint finale al **46%** del checkpoint ottimale (visibile in tabella 6.3), dimostrando come una tempestiva interruzione dell'addestramento (*early stopping*) sia fondamentale per preservare la capacità di astrazione della rete.

6.1.6 Confronto tra le configurazioni di addestramento

I risultati ottenuti confermano che la combinazione tra una generazione dati rigorosa e una strategia di fine-tuning ad alta efficienza parametrica consente a un modello di soli 270M parametri di raggiungere una discreta accuratezza.

6.1. TESTING

Configurazione di fine-tuning	Iterazioni	EM (In-Distr.)	EM (Out-Distr.)
LoRA Baseline (Short Run)	4000	13,0%	4,0%
Full Fine-Tuning	6000	18,0%	5,0%
LoRA Final Checkpoint	15000	15,0%	7,0%
LoRA Optimal (Min Val Loss)	7000	46,0%	14,0%

Tabella 6.3: Confronto comparativo dell'Exact Match (EM) tra le diverse configurazioni di addestramento su benchmark in-distribution ed out-of-distribution.

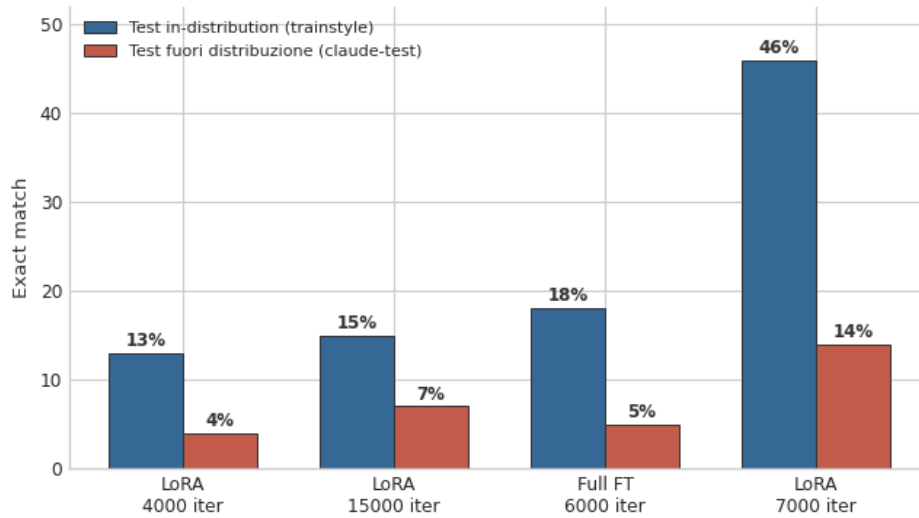


Figura 6.4: Confronto dell'Exact Match sulle configurazioni testate.

Come sintetizzato nella tabella 6.3, l'approccio basato su LoRA all'iterazione 7000 supera nettamente il Full Fine-Tuning condotto a parità di risorse (46% contro 18%). Il motivo di tale divario risiede nella tendenza del Full Fine-Tuning a destabilizzare le rappresentazioni latenti pre-addestrate del modello (*catastrophic forgetting*), mentre le matrici a basso rango di LoRA preservano le capacità grammaticali generali adattando unicamente l'interfaccia verso la sintassi SPARQL.

Sull'insieme out-of-distribution (*claudef-test*), i tassi di exact match risultano inferiori (4% - 7%), evidenziando la necessità di integrare in futuro un modulo di allineamento stilistico ad ampio raggio per domande formulate da sistemi terzi.

6.1.7 Tassonomia e analisi diagnostica degli errori

L'analisi qualitativa sui 54 casi di scarto registrati dal checkpoint ottimale (iterazione 7000) offre un riscontro importante sulla solidità del sistema (figura 6.5).

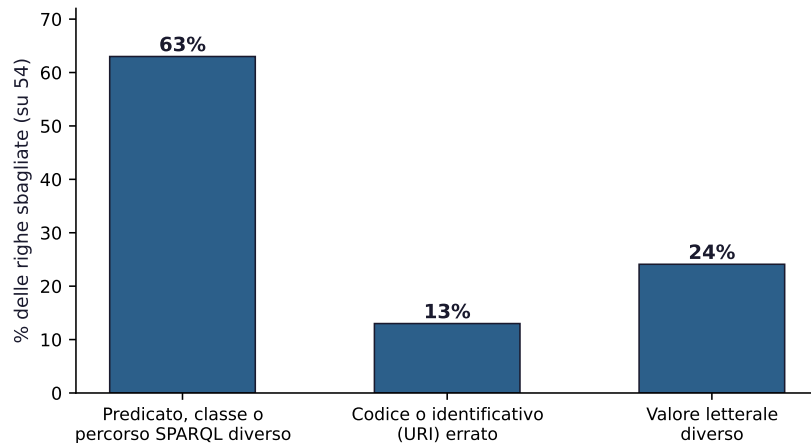


Figura 6.5: Distribuzione percentuale delle 54 predizioni non corrispondenti per tipologia di errore sul checkpoint ottimale (iterazione 7000).

I fallimenti sono stati categorizzati in tre tipologie strutturali:

1. **Deviazione ontologica del percorso (63%, 34 casi):** Il modello genera una query sintatticamente perfetta, ma impiega un predicato o una classe adiacente errata (es. inserimento arbitrario della sottoclasse `DrinkerPatient` in luogo del nodo generico `Patient`).
2. **Mancata corrispondenza dei valori letterali (24%, 13 casi):** La struttura di grafo e i predicati risultano esatti, ma uno dei valori di filtro differisce dal target di riferimento.
3. **Scambio di codici di dominio (13%, 7 casi):** La query individua la corretta classe, ma confonde due identificativi controllati specifici (es. inversione di due codici di patologia ICD-10 entrambi plausibili nel contesto).

Il dato più rilevante emerso dall'analisi è che il **100% delle query generate è sintatticamente valido**. In nessun caso il modello ha prodotto parentesi spaiate, keyword SPARQL errate o violazioni di grammatica. Ciò dimostra che la pipeline di addestramento ha insegnato con successo la struttura rigida del linguaggio di interrogazione, riducendo l'incertezza residua a una semplice discriminazione tra entità ontologiche contigue.

Capitolo 7

Conclusioni e sviluppi futuri

7.1 Sviluppi futuri

7.1.1 Risoluzione dei problemi di qualità

Alcuni dei problemi di qualità discussi in sezione 6.1.1 non hanno ricevuto, nell'ambito di questo lavoro, una correzione alla radice, e rappresentano quindi possibili direzioni di sviluppo futuro.

- **Confusione tra acronimi clinici simili** (sezione 6.1.1). Una possibile correzione consisterebbe nel restringere la generazione di sinonimi per questa categoria di valori a un dizionario curato di equivalenze verificate, invece di affidarsi alla generazione libera del modello linguistico locale, che non dispone di una base affidabile di terminologia clinica.
- **Allucinazione di codici ICD-10** (sezione 6.1.1). Il difetto specifico nel filtro di riconoscimento è già stato corretto, ma una correzione più robusta consisterebbe nell'escludere del tutto dalla generazione libera qualunque valore riconducibile a un sistema di codifica controllato, sostituendola con una verifica rispetto ad una tabella dei codici validi prima di accettare qualunque sostituzione proposta, così da eliminare il rischio alla radice invece di limitarsi a riconoscere meglio i pattern da escludere.

- **Valore letterale "nan" come unità di misura** (sezione 6.1.1). Una correzione consisterebbe nel riconoscere esplicitamente questo tipo di valore mancante durante la generazione del dataset strutturato, escludendo il record interessato o rappresentando l'assenza del dato in modo esplicito, invece di lasciare che un segnaposto tecnico non gestito raggiunga il testo finale.

7.1.2 Altre direzioni di sviluppo

Oltre alla risoluzione dei problemi di qualità appena discussi, si possono proporre alcune direzioni di sviluppo più ampie, emerse dai limiti riscontrati soprattutto nel corso della valutazione.

- **Valutazione a livello di esecuzione.** La metrica di valutazione impiegata in questo lavoro, l'*exact match*, richiede una corrispondenza carattere per carattere tra la query generata e quella di riferimento. Una query semanticamente equivalente ma formulata in modo sintatticamente diverso, ad esempio con clausole in un ordine differente, viene quindi conteggiata come errore anche quando produrrebbe lo stesso risultato se eseguita sul knowledge graph. Una valutazione complementare, che esegua le query generate sull'endpoint di CVL e ne confronti i risultati restituiti anziché il solo testo, fornirebbe una stima più accurata delle prestazioni reali del modello.
- **Altre tecniche o modelli di adattamento.** Questo lavoro ha confrontato LoRA con il full fine-tuning, ma non ha testato QLoRA, nonostante fosse introdotta in sezione 2.5.3 come tecnica in grado di ridurre ulteriormente l'occupazione di memoria rispetto a LoRA; allo stesso modo, non sono stati testati modelli di base diversi da **Gemma3:270M**, di dimensioni maggiori o architettura differente, che potrebbero offrire una migliore capacità di generalizzazione a scapito di un costo computazionale più elevato.
- **Estensione alla lingua italiana.** L'attuale pipeline produce varianti conversazionali esclusivamente in lingua inglese, in allineamento con la nomenclatura dell'ontologia di base. Uno sviluppo futuro di grande impatto applicativo consisterebbe nell'estendere il modulo di *augmentation* per generare e validare registri stilistici direttamente in italiano. La creazione di un corpus

di addestramento localizzato abiliterebbe il *fine-tuning* di modelli capaci di tradurre interrogazioni mediche formulate nativamente in italiano in query SPARQL, abbattendo le barriere linguistiche per gli utenti finali e facilitando l'adozione del sistema da parte del personale clinico nel contesto nazionale.

7.1.3 Conclusioni

Questo lavoro di tesi ha affrontato e risolto l'assenza di un dataset di addestramento specifico per l'interrogazione in linguaggio naturale del *knowledge graph* del Cancer Virtual Lab. A tale scopo, è stata progettata, sviluppata e validata una pipeline automatica a due stadi per la generazione dei dati, la cui efficacia è stata dimostrata empiricamente attraverso il *fine-tuning* di un modello linguistico ultraleggero.

I quattro obiettivi architetturali stabiliti in fase di analisi (capitolo 3) sono stati pienamente soddisfatti:

- **O1 - Copertura selettiva ed estendibilità:** garantita dall'introduzione di un sistema a *tier*, che mappa con successo le entità ontologiche selezionate con facilità. Il sistema risulta nativamente estendibile a nuove classi tramite configurazioni esterne, azzerando la necessità di interventi sul codice sorgente (sezione 5.1.11).
- **O2 - Varietà linguistica autentica:** ottenuta mediante un modulo di *augmentation* capace di declinare ogni interrogazione in dieci registri stilistici differenti, mantenendo un tasso di successo superiore al 97% persino nelle configurazioni di mascheramento più restrittive (sezione 6.1.2).
- **O3 - Tracciabilità completa:** assicurata dalla propagazione di un identificativo univoco di provenienza lungo tutta la pipeline, questo meccanismo ha permesso di certificare matematicamente l'assenza di *data leakage* tra gli insiemi di *train*, *validation* e *test* (sezione 6.1.3).
- **O4 - Volume di dati adeguato:** validato non solo dalla produzione massiva di 14021 varianti, ma soprattutto dall'efficacia pratica del dataset, rivelatosi determinante per l'addestramento e il raggiungimento di risultati quantificabili sul modello linguistico finale.

Sotto il profilo empirico, la valutazione del modello ha restituito risultati di assoluto rilievo. L'applicazione della tecnica LoRA su un'architettura ultra-compatta di soli 270 milioni di parametri ha permesso di raggiungere un *Exact Match* del 46% su un benchmark in stile coerente con i dati di addestramento.

Ancora più rilevante è l'esito dell'analisi degli errori, in cui in nessun caso il modello ha generato query sintatticamente invalide. Possiamo dunque dedurre che la rete ha interiorizzato perfettamente la complessa grammatica SPARQL, confidando le incertezze residue alla sola discriminazione semantica tra entità o valori ontologici simili, escludendo così derive allucinatorie o comportamenti arbitrari.

Infine, il rigoroso approccio iterativo adottato ha consentito di identificare, diagnosticare e documentare con precisione cinque anomalie qualitative nel dataset generato (sezione 6.1.1). Avendo ricondotto ciascuna casistica alla rispettiva radice tecnica, due di queste criticità sono state risolte in modo definitivo all'interno di questo lavoro, mentre per le restanti tre è stato tracciato un chiaro percorso di risoluzione per gli sviluppi futuri. Parallelamente, un'analisi approfondita ha permesso di riqualificare due comportamenti apparentemente anomali come variazioni fisiologiche e legittime.

In prospettiva, i risultati di questa ricerca dimostrano che l'uso di modelli ultra-compatti su dati clinici, imposto dai vincoli di privacy, non rappresenta un limite architetturale invalicabile. Se supportati da dati sintetici strutturati di altissima qualità e da metodologie di adattamento mirate, questi modelli possono superare la semplice validazione sintattica, arrivando a interrogare con successo la complessa semantica di un dominio medico specialistico.

Bibliografia

- [AAG⁺25] P. De Angelis, A. Andalò, N. Gentili, L. Giorgetti, L. Ridolfi, R. Pasolini, A. Pagliarani, M. Cavallucci, R. Vespignani, and Antonella Carbonaro. Cancer virtual lab: una piattaforma sicura e interoperabile basata su knowledge graph e large language model per la ricerca oncologica. *Recenti Progressi in Medicina*, 116(10):601–602, 2025.
- [CGR⁺25] Antonella Carbonaro, L. Giorgetti, L. Ridolfi, R. Pasolini, A. Pagliarani, M. Cavallucci, A. Andalò, L. Del Gaudio, P. De Angelis, R. Vespignani, and N. Gentili. From raw data to research-ready: A FHIR-based transformation pipeline in a real-world oncology setting. *Computers in Biology and Medicine*, 197:111051, 2025.
- [DBAL19] Mohnish Dubey, Debayan Banerjee, Abdelrahman Abdelkawi, and Jens Lehmann. Lc-quad 2.0: A large dataset for complex question answering over wikidata and dbpedia. In *The Semantic Web – ISWC 2019*, pages 69–78. Springer, 2019.
- [HSW⁺22] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [LdCF25] A. B. Labate, B. W. S. R. de Carvalho, and S. R. Fiorini. Synthetic dataset generation and validation with LLMs for text2sparql in Wikidata graphs. In *Proceedings of the Wikidata Workshop 2025*, 2025.

- [SCKS20] B. A. Spiegel, V. Cheong, J. E. Kaplan, and A. Sanchez. MK-SQuIT: Synthesizing questions using iterative template-filling, 2020.
- [TMDL17] Priyansh Trivedi, Gaurav Maheshwari, Mohnish Dubey, and Jens Lehmann. Lc-quad: A corpus for complex question answering over knowledge graphs. In *The Semantic Web – ISWC 2017*, pages 210–218. Springer, 2017.